

本节内容

树

存储结构

知识总览

树的存储结构

树的逻辑结构回顾

双亲表示法（顺序存储）

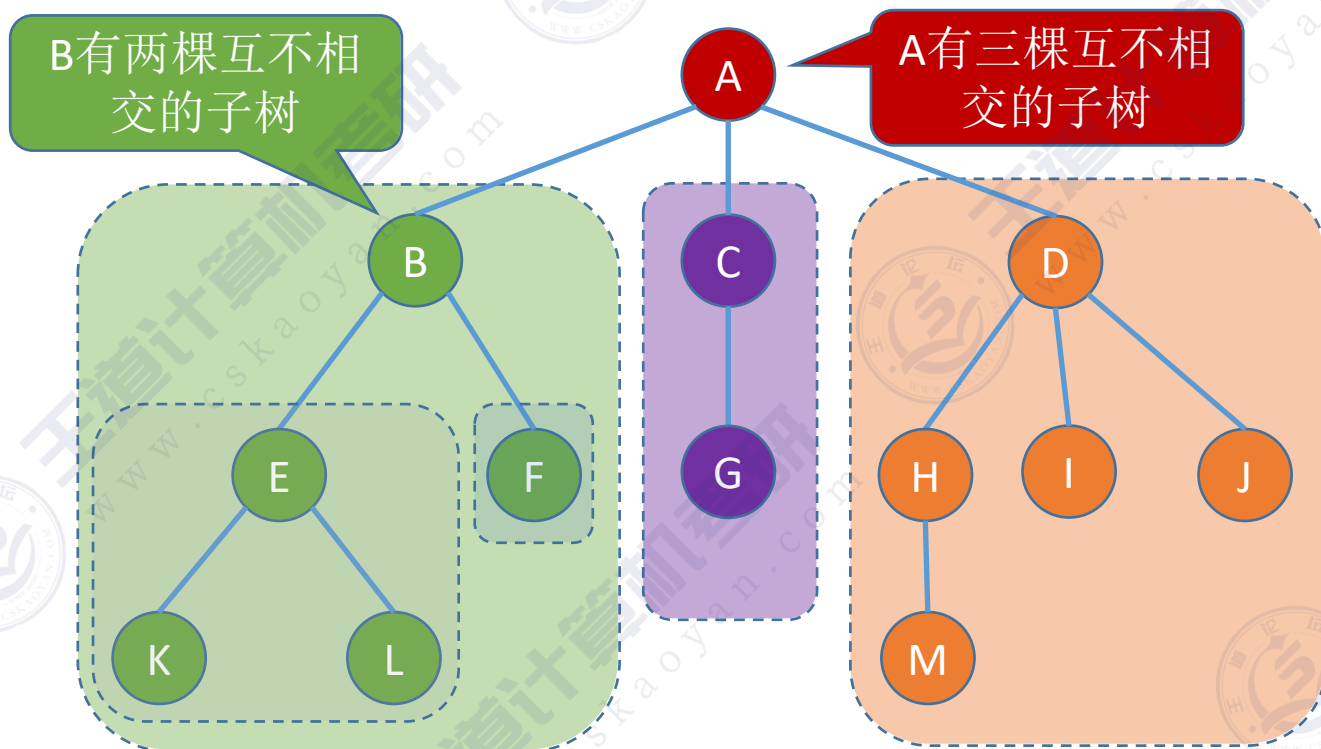
孩子表示法（顺序+链式存储）

孩子兄弟表示法（链式存储）

树的逻辑结构

树是 n ($n \geq 0$) 个结点的有限集合, $n = 0$ 时, 称为空树, 这是一种特殊情况。在任意一棵非空树中应满足:

- 1) 有且仅有一个特定的称为根的结点。
- 2) 当 $n > 1$ 时, 其余结点可分为 m ($m > 0$) 个互不相交的有限集合 $T_1, T_2, \dots, T_m$, 其中每个集合本身又是一棵树, 并且称为根结点的子树。

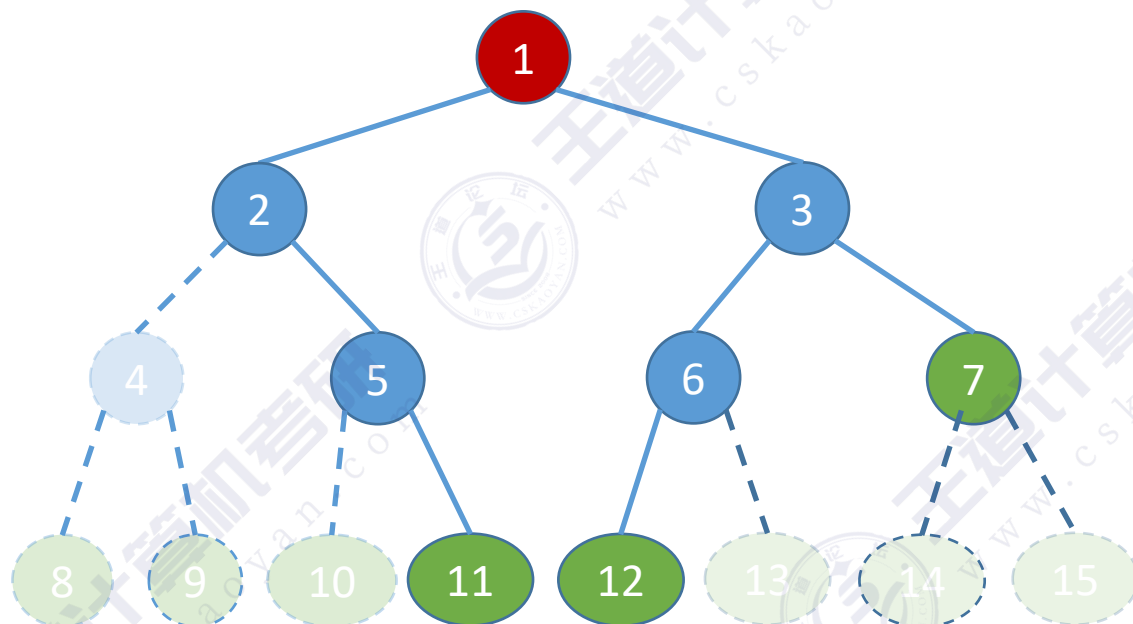


🌲 树是一种递归定义的数据结构

二叉树: 一个分支结点最多只能有两棵子树

树: 一个分支结点可以有多棵子树

回顾：二叉树的顺序存储



二叉树：一个分支结点最多只能有两棵子树

二叉树的顺序存储：

按照完全二叉树中的结点顺序，将各结点存储到数组的对应位置。数组下标反映结点之间的逻辑关系

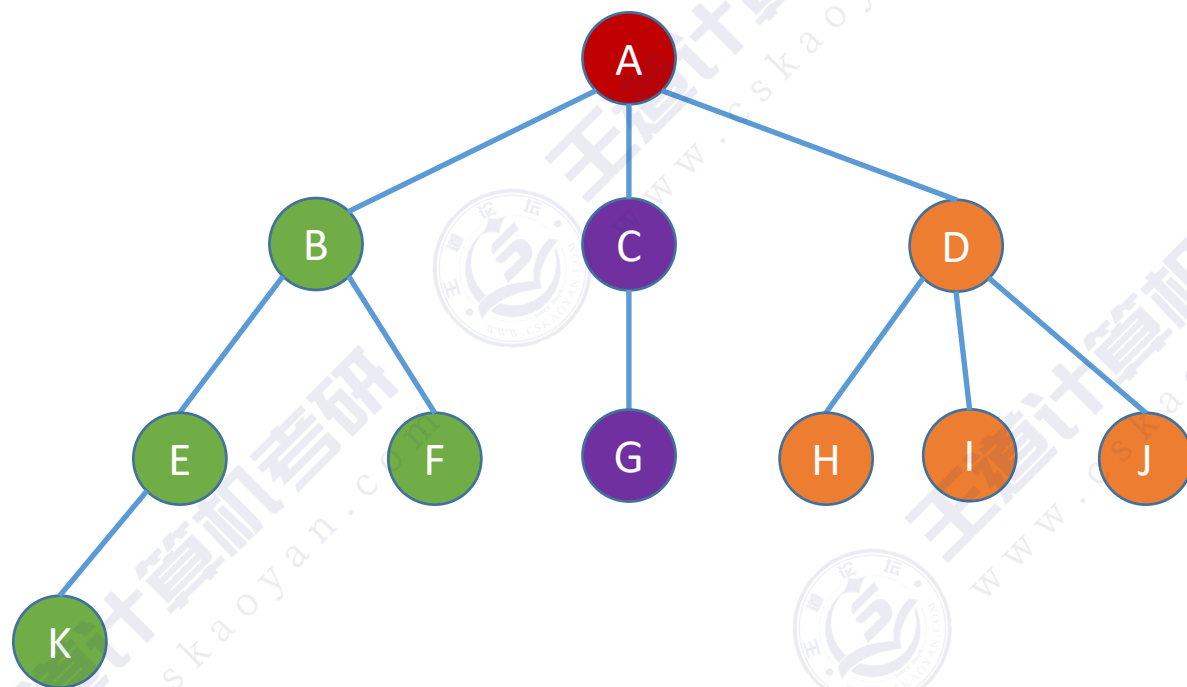
若根结点从数组下标1开始存放，则：

- i 的左孩子 —— $2i$
- i 的右孩子 —— $2i+1$
- i 的父节点 —— $\lfloor i/2 \rfloor$



t[0] t[1] t[2]

如何实现树的顺序存储?



树：一个分支结点可以有多棵子树

只依靠数组下标，无法反映结点之间的逻辑关系

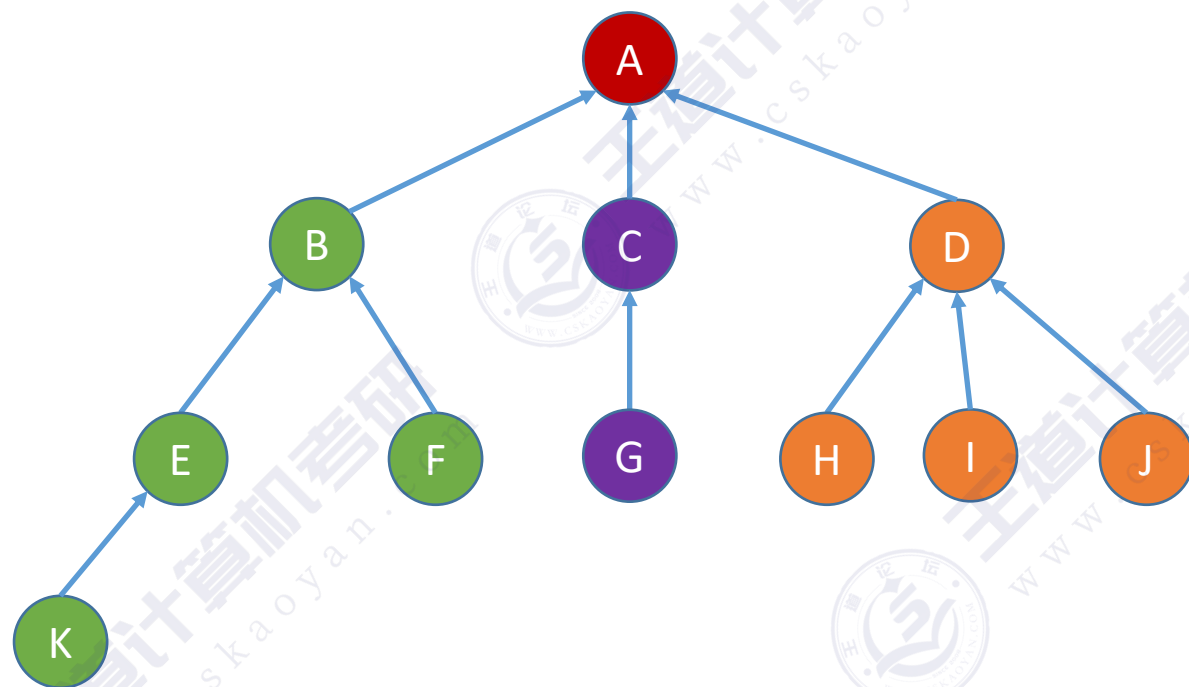


行不通的，笨蛋



t[0] t[1] t[2]

如何实现树的顺序存储？



思路：用数组顺序存储各个结点。每个结点中保存数据元素、指向双亲结点（父结点）的“指针”

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	G	2
7	H	3
8	I	3
9	J	3
10	K	4
11		
12		
13		

根节点的双亲指针 = -1

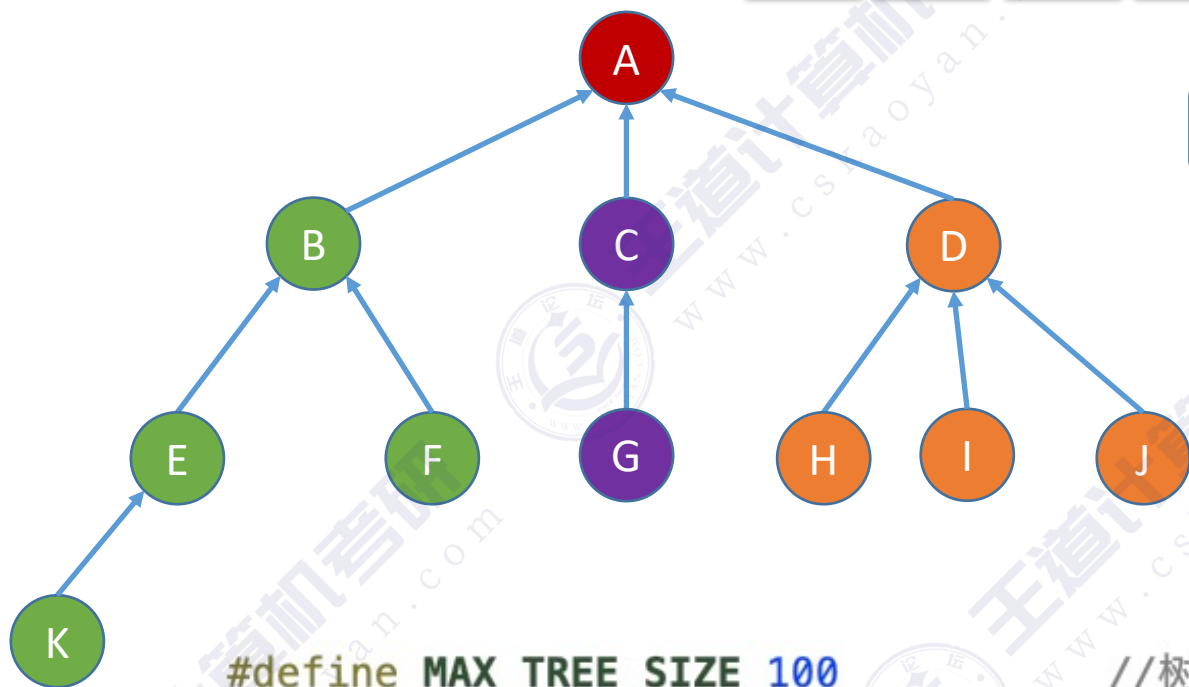
非根节点的双亲指针 = 父节点在数组中的下标

⋮

⋮

树的存储1: 双亲表示法

顺序存储



```
#define MAX_TREE_SIZE 100
```

```
typedef struct{
    ElemType data;
    int parent;
}PTNode;
```

```
typedef struct{
    PTNode nodes[MAX_TREE_SIZE];
    int n;
}PTree;
```

结点总数n=11

//树中最多结点数

//树的结点定义

//数据元素

//双亲位置域

//树的类型定义

//双亲表示

//结点数

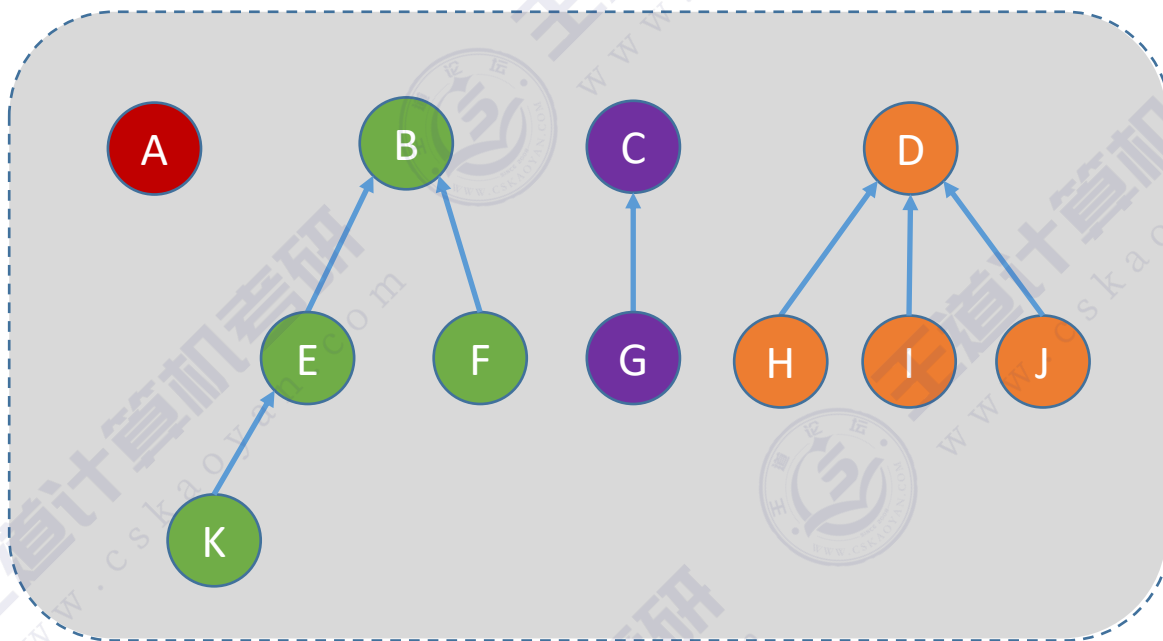
	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	G	2
7	H	3
8	I	3
9	J	3
10	K	4
11		
12		
13		

⋮

⋮

拓展：双亲表示法存储“森林”

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合



森林

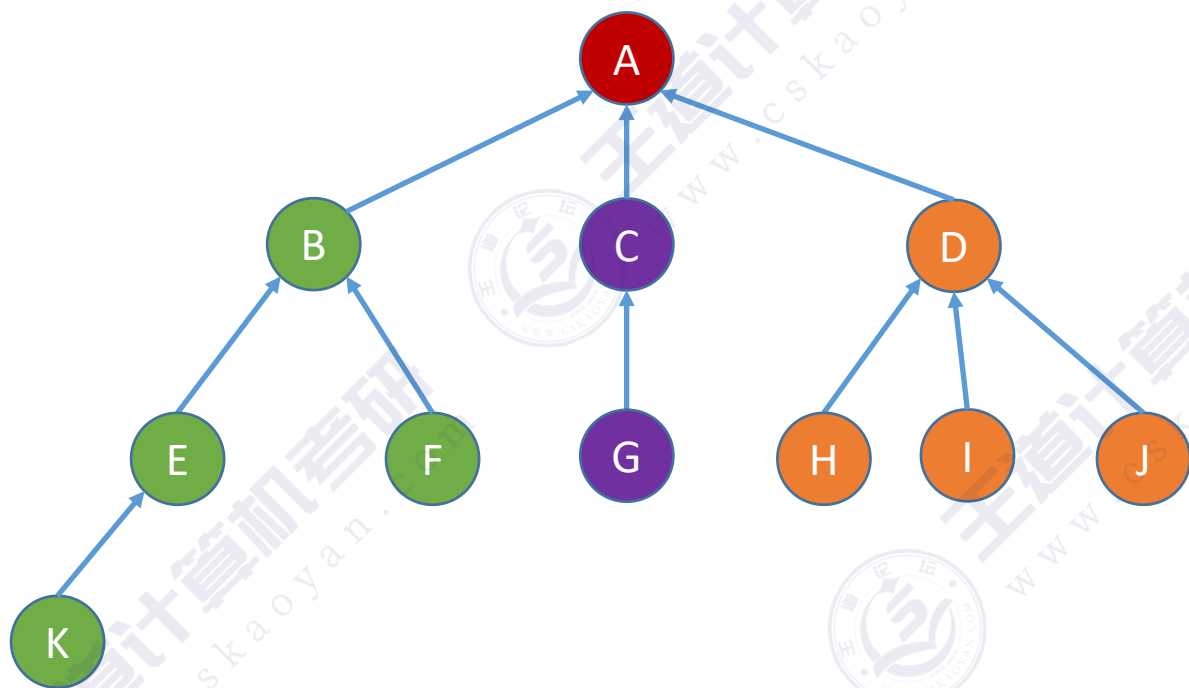
	data	parent
0	A	-1
1	B	-1
2	C	-1
3	D	-1
4	E	1
5	F	1
6	G	2
7	H	3
8	I	3
9	J	3
10	K	4
11		
12		
13		

每棵树的根
节点双亲指
针 = -1

⋮

⋮

双亲表示法的优缺点



p →

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	G	2
7	H	3
8	I	3
9	J	3
10	K	4
11		
12		
13		

4号结点的
父节点是1
号结点

10号结点是4
号结点的孩子

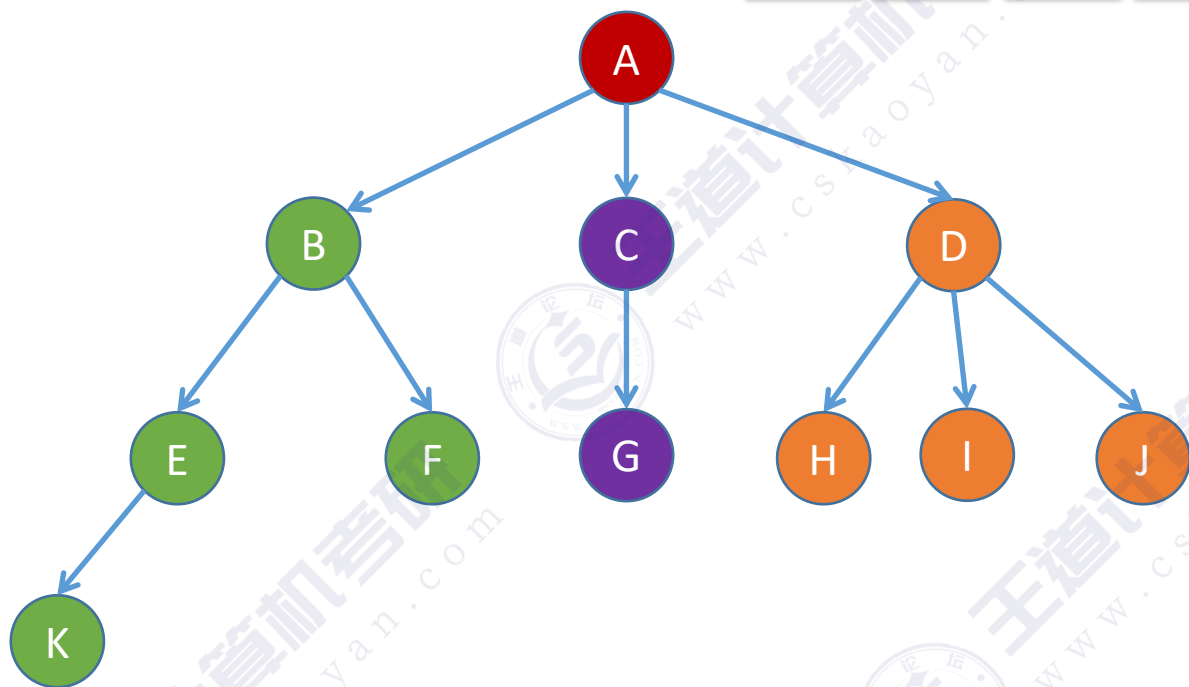
双亲表示法

优点：找双亲（父节点）很方便

缺点：找孩子不方便，只能从头到尾遍历整个数组

适用于“找父亲”多，“找孩子”少的应用场景。如：并查集

树的存储2：孩子表示法



孩子表示法：用数组顺序存储各个结点。每个结点中保存数据元素、孩子链表头指针

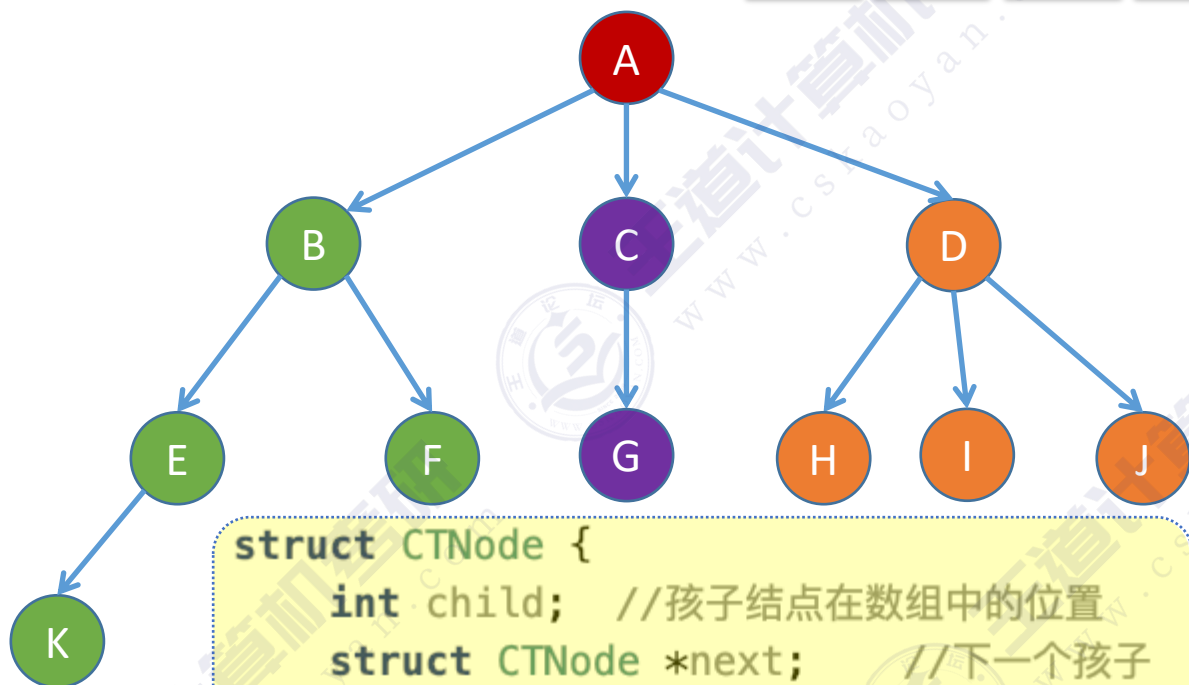
	data	*firstChild
0	A	1 → 2 → 3 ^
1	B	4 → 5 ^
2	C	6 ^
3	D	7 → 8 → 9 ^
4	E	10 ^
5	F	^
6	G	^
7	H	^
8	I	^
9	J	^
10	K	^
	⋮	⋮

指向第一个孩子编号

用一个链表记录一个结点的孩子编号

树的存储2: 孩子表示法

顺序存储+链式存储结合



```
struct CTNode {  
    int child; //孩子结点在数组中的位置  
    struct CTNode *next; //下一个孩子  
};
```

```
typedef struct {  
    ElemType data;  
    struct CTNode *firstChild; //第一个孩子  
} CTBox;
```

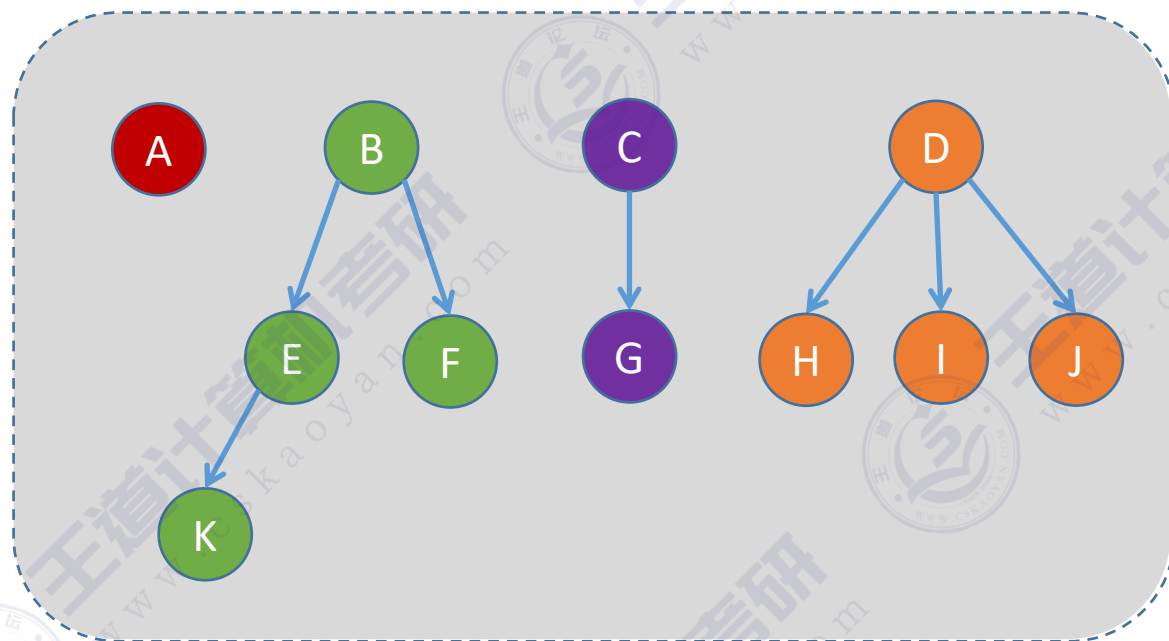
```
typedef struct {  
    CTBox nodes[MAX_TREE_SIZE];  
    int n, r; //结点数和根的位置  
} CTree;
```

结点总数n=11, 根的位置r=0

	data	*firstChild
0	A	1 → 2 → 3 ^
1	B	4 → 5 ^
2	C	6 ^
3	D	7 → 8 → 9 ^
4	E	10 ^
5	F	^
6	G	^
7	H	^
8	I	^
9	J	^
10	K	^
:	:	:

拓展：孩子表示法存储“森林”

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合

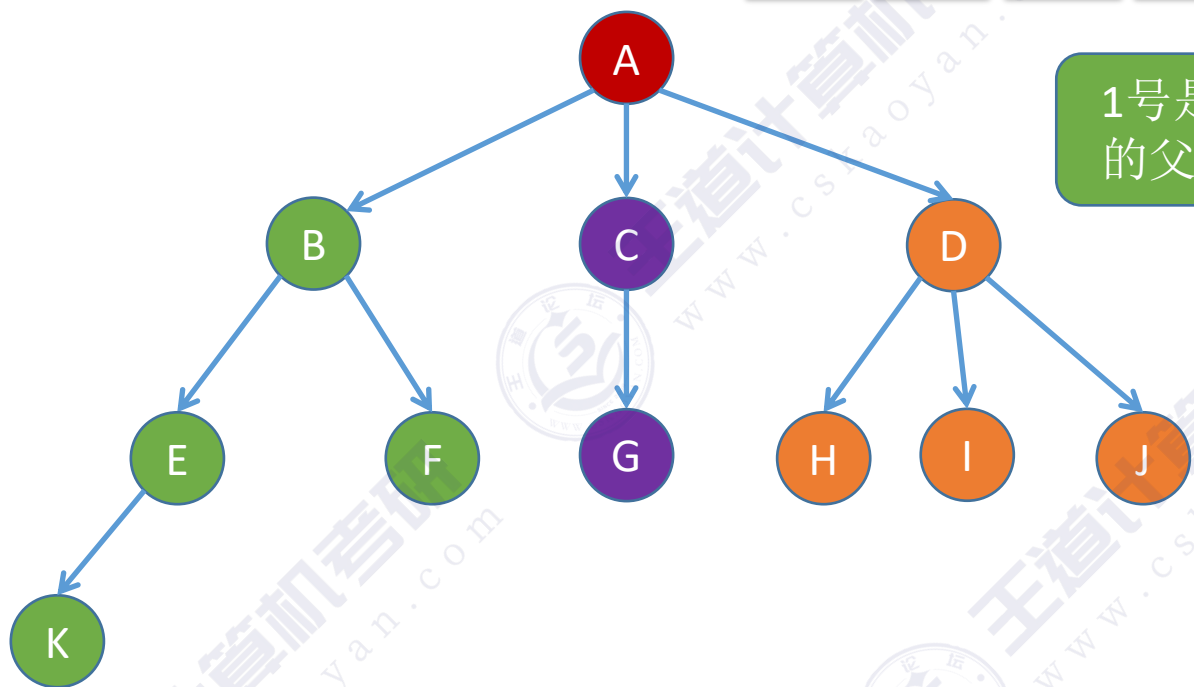


森林

注：用孩子表示法存储森林，需要记录多个根的位置

	data	*firstChild
0	A	^
1	B	→ 4
2	C	→ 6
3	D	→ 7
4	E	→ 10
5	F	^
6	G	^
7	H	^
8	I	^
9	J	^
10	K	^
⋮		⋮

孩子表示法的优缺点



1号是4号的父结点

p

	data	*firstChild
0	A	
1	B	
2	C	
3	D	
4	E	
5	F	^
6	G	^
7	H	^
8	I	^
9	J	^
10	K	^
:	:	:

10号是4号的孩子结点

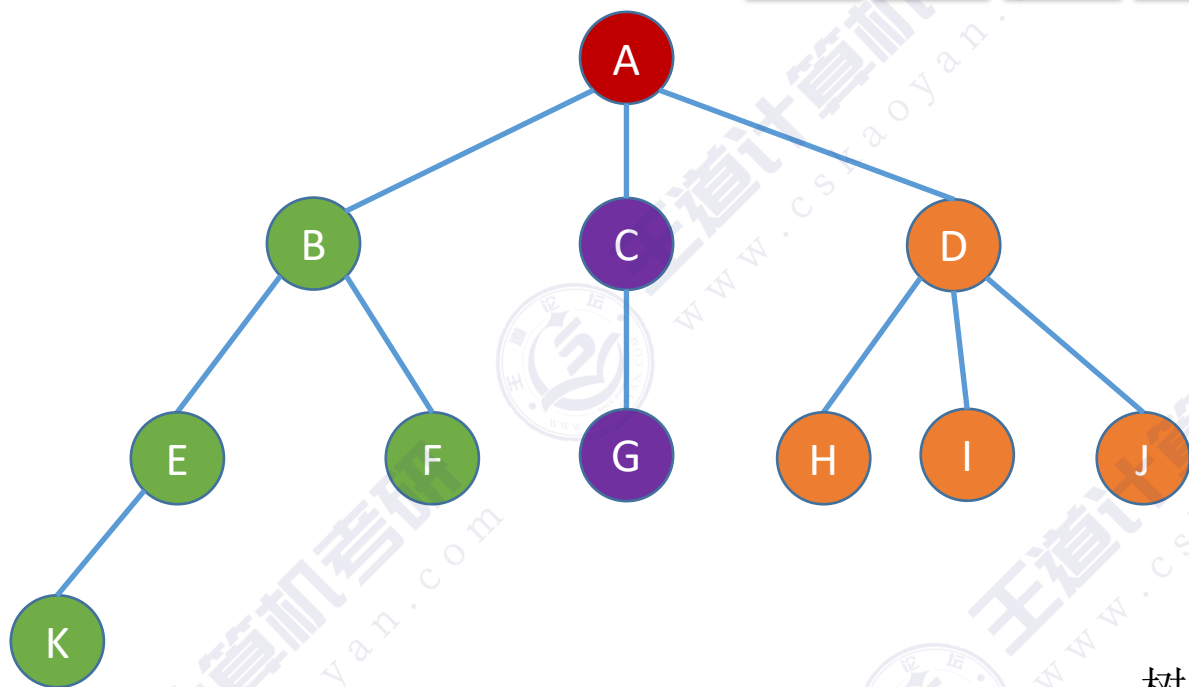
孩子表示法

优点：找孩子很方便

缺点：找双亲（父节点）不方便，只能遍历每个链表

适用于“找孩子”多，“找父亲”少的应用场景。如：服务流程树

树的存储3：孩子兄弟表示法



//二叉树的结点（链式存储）

```
typedef struct BiTNode{
    ElemType data;
    struct BiTNode *lchild, *rchild;
}BiTNode, *BiTree;
```

左子树

右子树

树的**孩子兄弟表示法**，与二叉树类似，采用**二叉链表**实现。每个结点内保存**数据元素**和**两个指针**，但两个指针的含义与二叉树结点不同

//树的存储——孩子兄弟表示法

```
typedef struct CSNode{
    ElemType data;
    struct CSNode *firstchild, *nextsibling;
}CSNode, *CSTree;
```

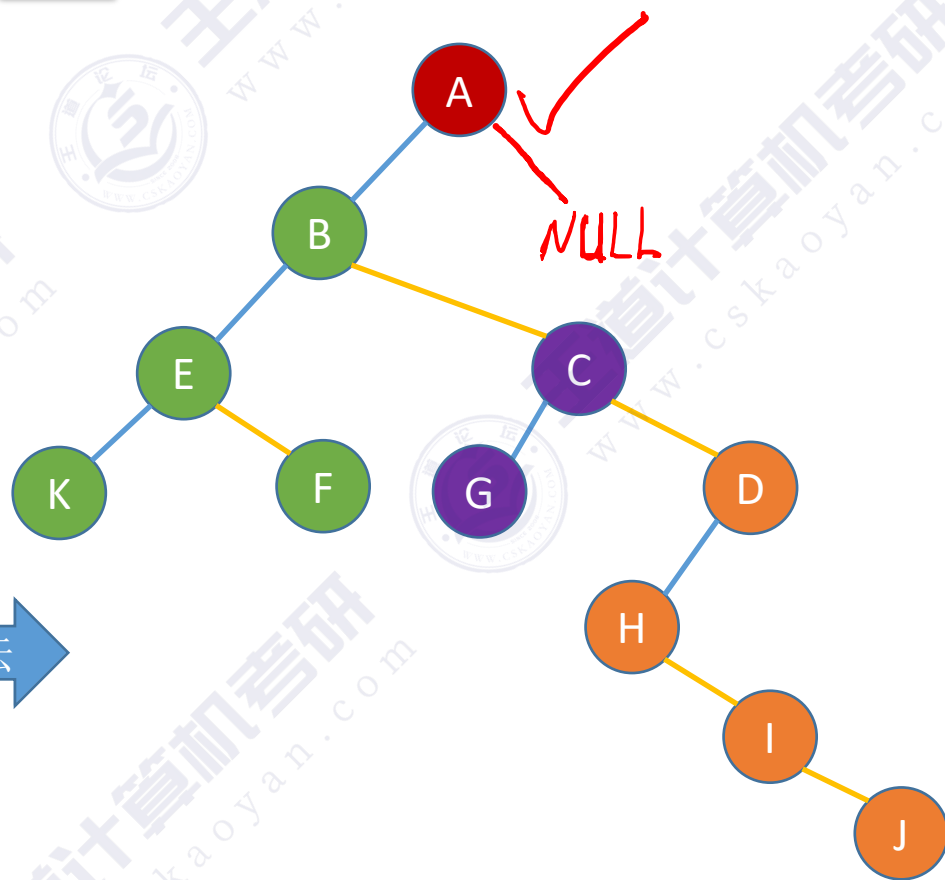
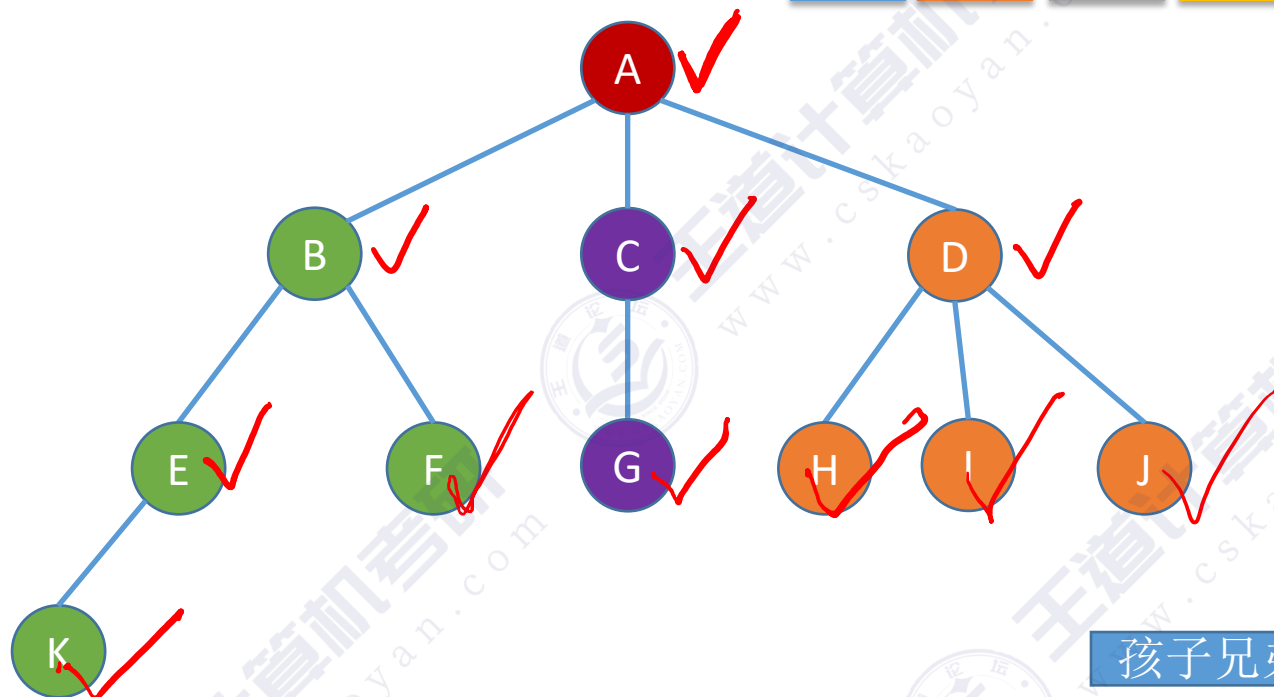
指向第一个孩子

指向右边一个兄弟

//数据域

//第一个孩子和右兄弟指针

树的存储3: 孩子兄弟表示法



//树的存储—孩子兄弟表示法

```
typedef struct CSNode{  
    ElemType data;  
    struct CSNode *firstchild, *nextsibling;  
}CSNode, *CSTree;
```

指向第一个孩子

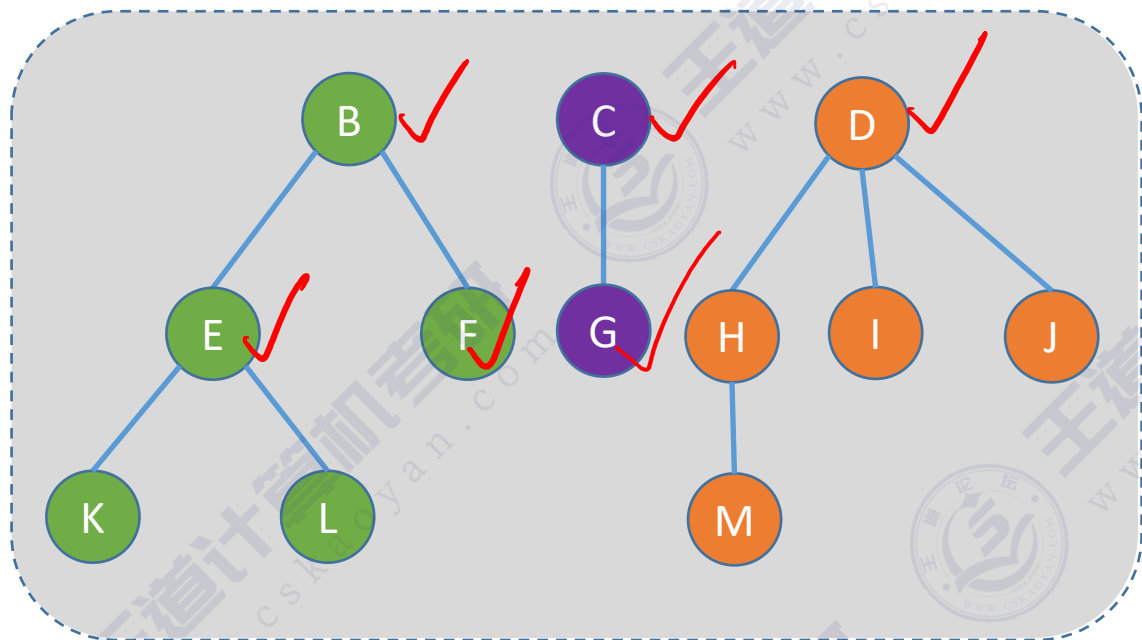
指向右边一个兄弟

//数据域

//第一个孩子和右兄弟指针

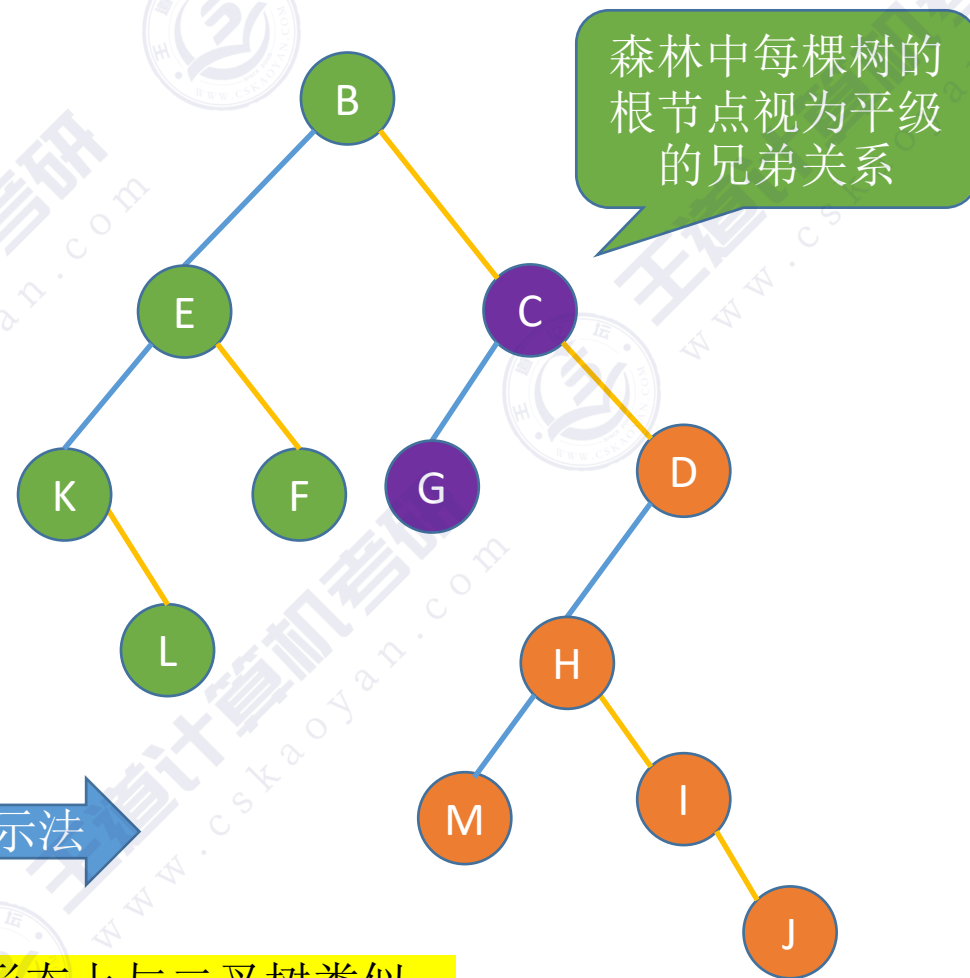
拓展：孩子兄弟表示法存储“森林”

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合



森林

孩子兄弟表示法



当使用“孩子兄弟表示法”存储树或森林时，从存储视角来看形态上与二叉树类似。

知识回顾与重要考点

树、森林的存储结构

双亲表示法

- 顺序存储结点数据，结点中保存父节点在数组中的下标
- 优点：找父节点方便；缺点：找孩子不方便

孩子表示法

- 顺序存储结点数据，结点中保存孩子链表头指针（顺序+链式存储）
- 优点：找孩子方便；缺点：找父节点不方便

★ 孩子兄弟表示法

- ★ 用二叉链表存储结点——左孩子右兄弟
- ★ 用于存储森林时，将森林中每棵树的根节点视为平级的兄弟关系
- ★ 从存储视角来看形态上和二叉树类似

重要考点：树、森林与二叉树的转换

本节内容

树、森林 与二叉树的转换

知识总览

树、森林与二叉树的转换

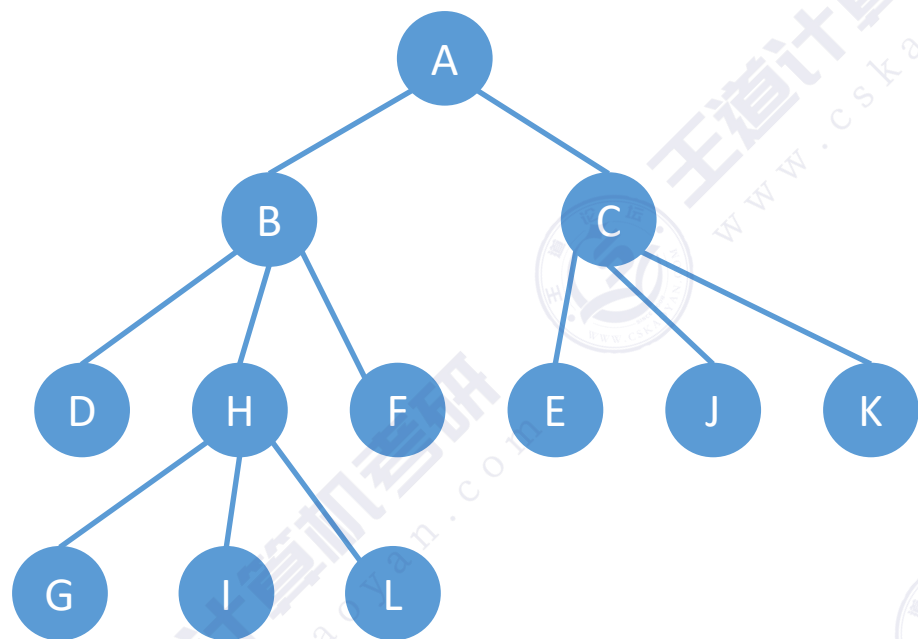
树 转 二叉树

森林 转 二叉树

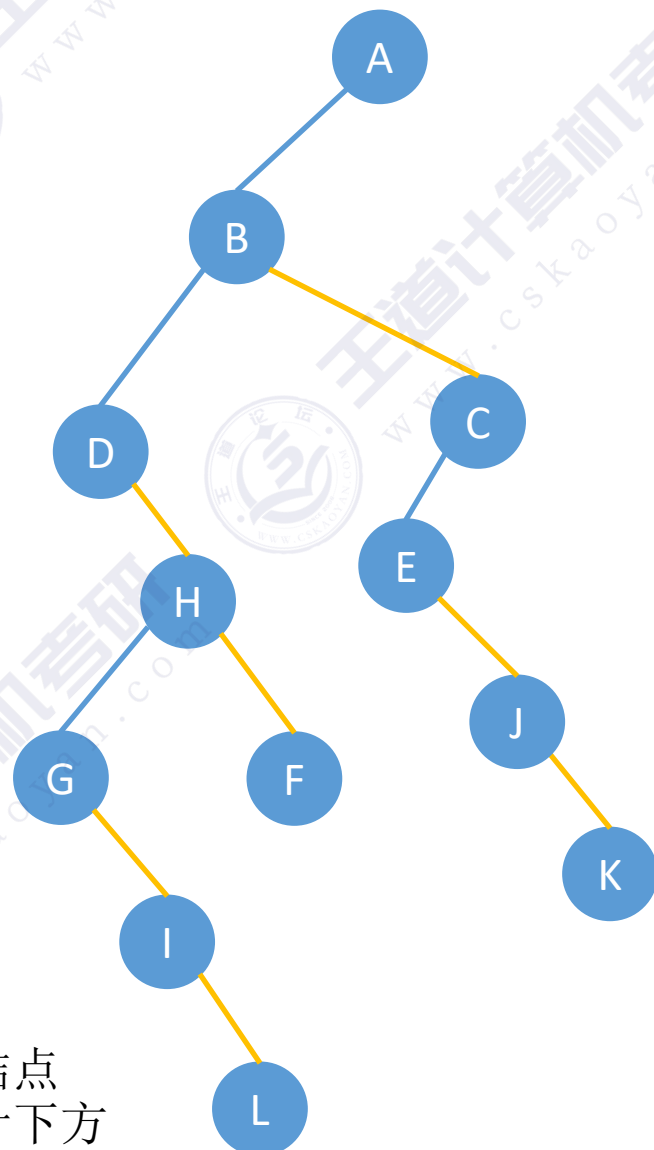
二叉树 转 树

二叉树 转 森林

树→二叉树 的转换



孩子兄弟表示法

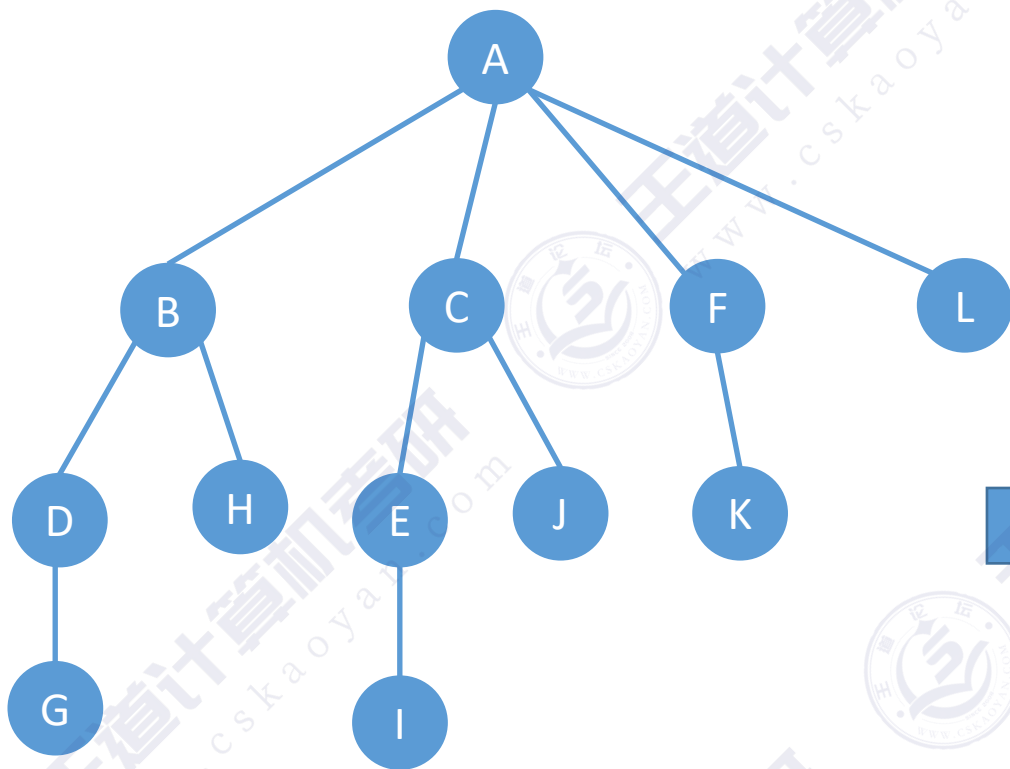


树→二叉树 转换技巧:

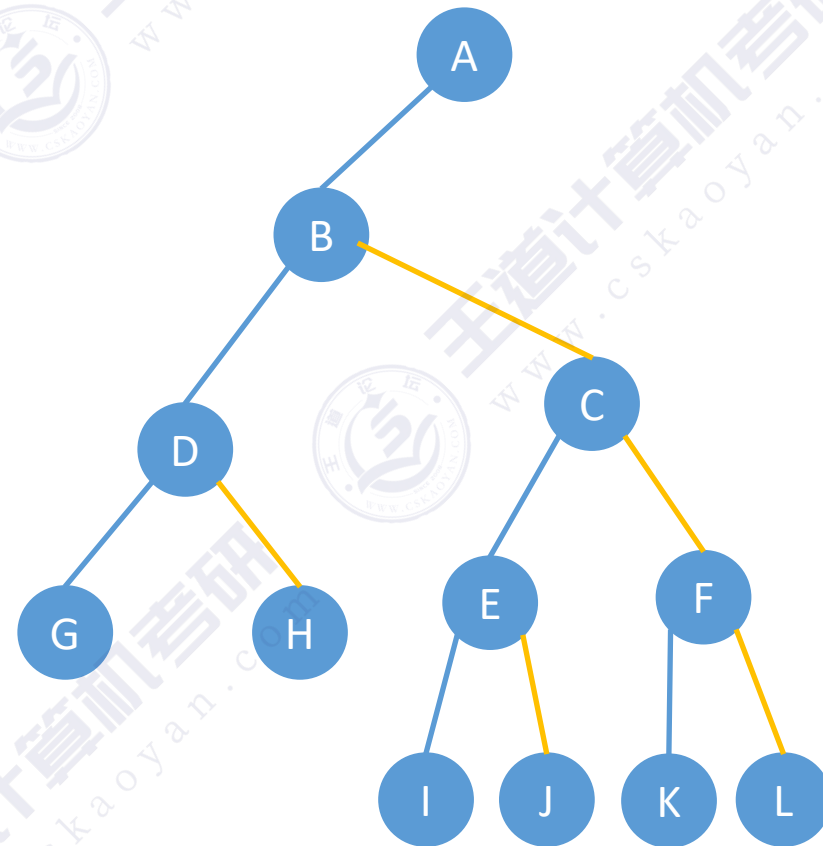
- ①先在二叉树中，画一个根节点。
- ②按“树的层序”依次处理每个结点。

处理一个结点的方法是：如果当前处理的结点在树中有孩子，就把所有孩子结点“用右指针串成糖葫芦”，并在二叉树中把第一个孩子挂在当前结点的左指针下方

树→二叉树 的转换

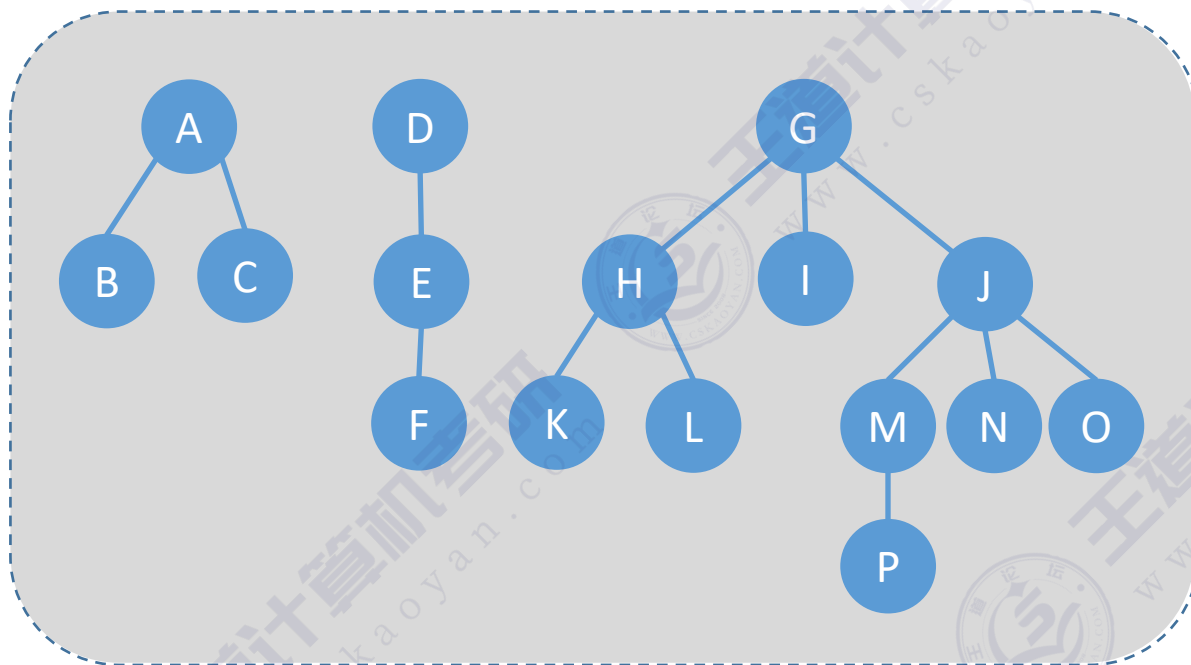


孩子兄弟表示法

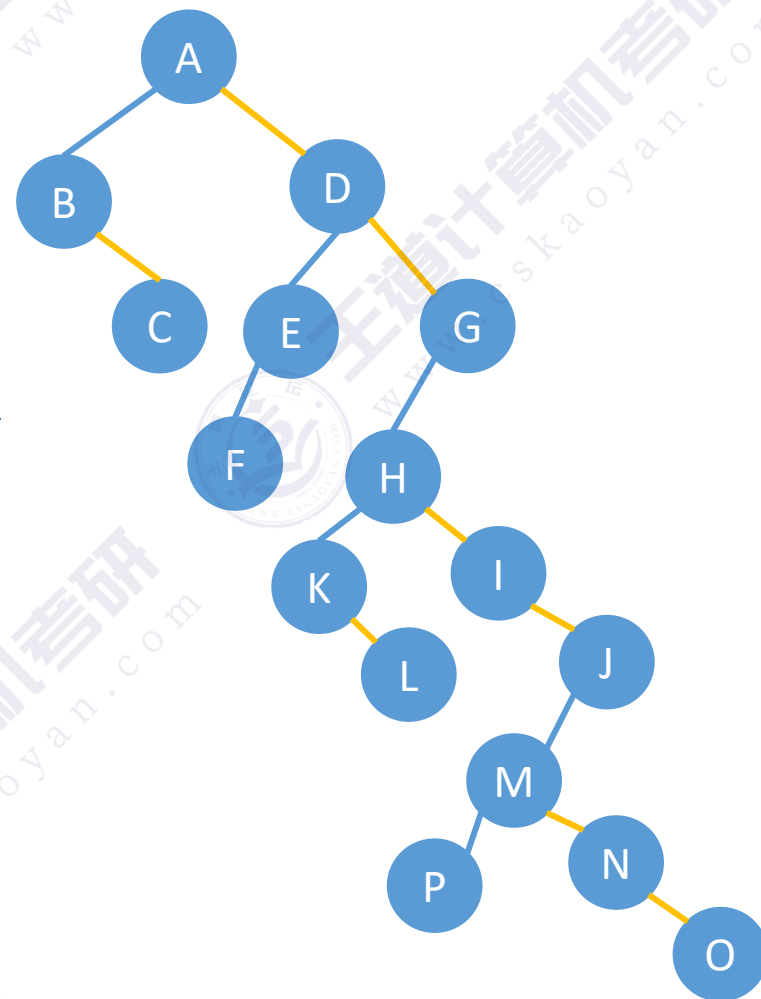


森林→二叉树 的转换

注意：森林中各棵树的根节点视为平级的兄弟关系



孩子兄弟表示法



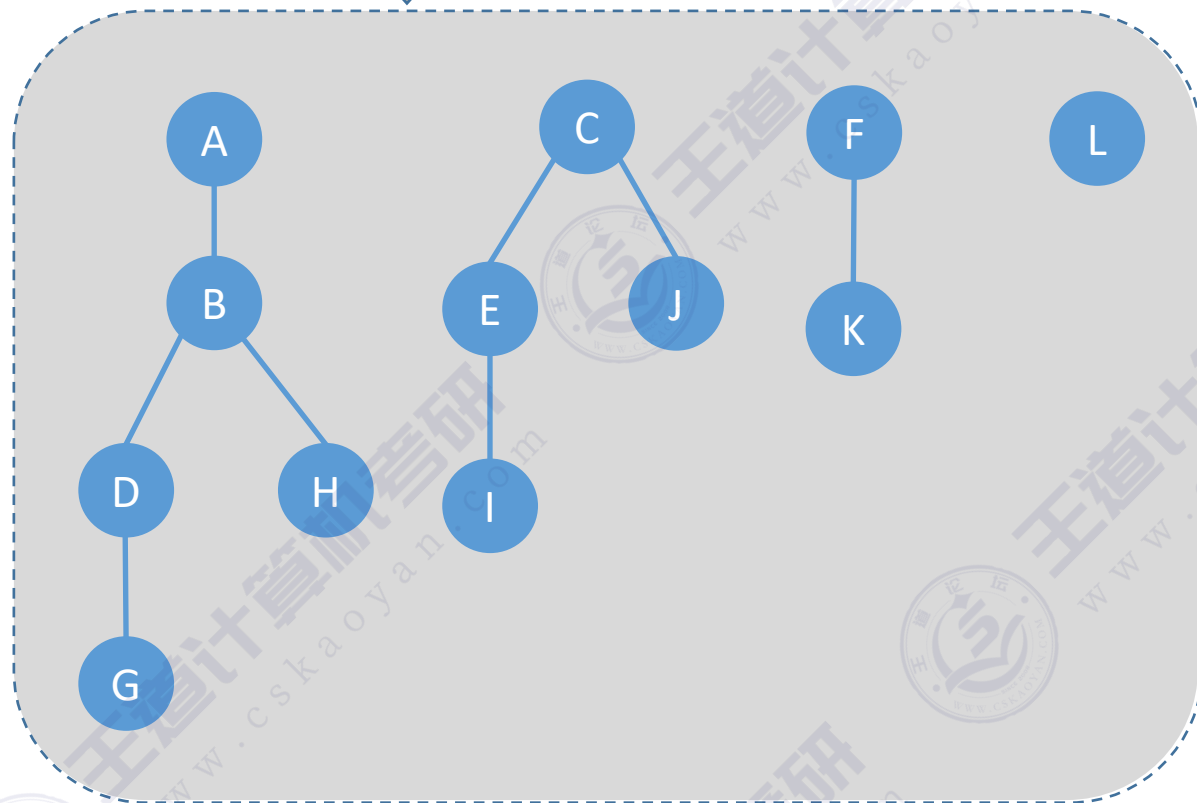
森林→二叉树 转换技巧：

- ①先把所有树的根结点画出来，在二叉树中用右指针**串成糖葫芦**。
- ②按“**森林的层序**”依次处理每个结点。

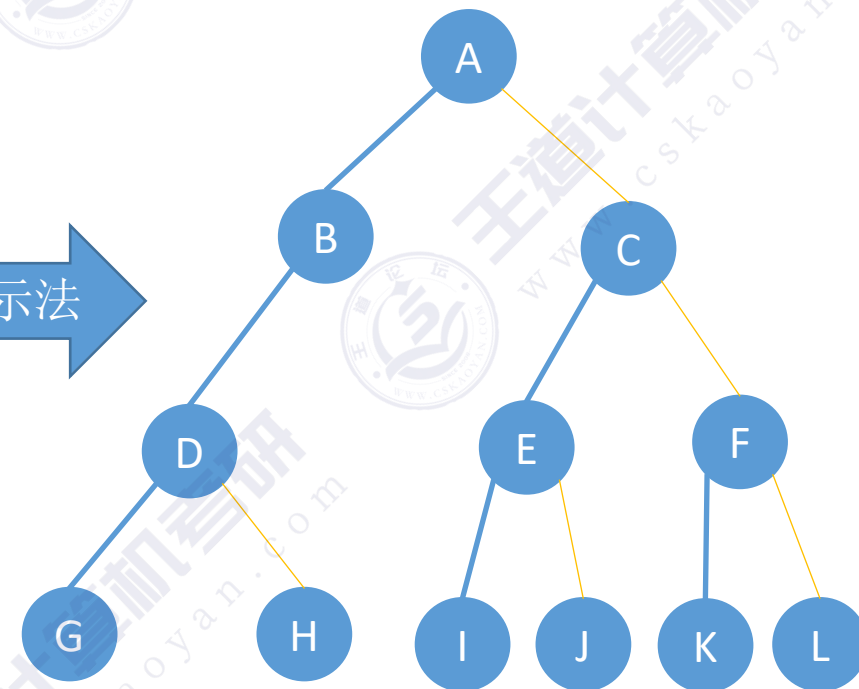
处理一个结点的方法是：如果当前处理的结点在树中有孩子，就把所有孩子结点“用右指针串成糖葫芦”，并在二叉树中把第一个孩子挂在当前结点的左指针下方

森林→二叉树的转换

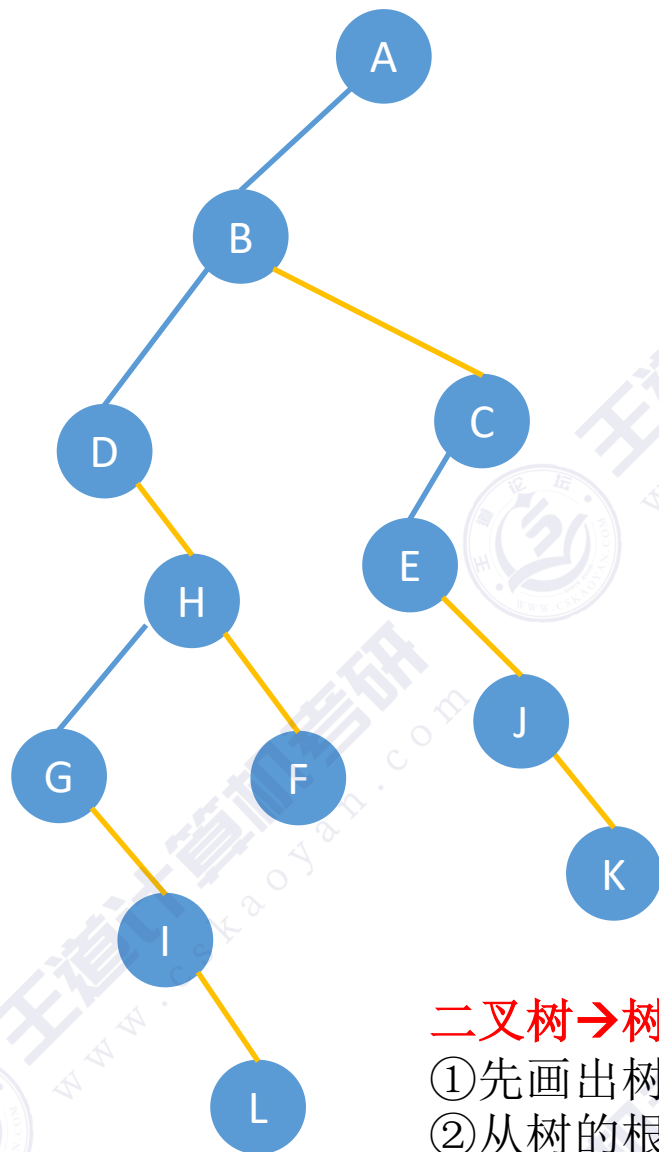
森林



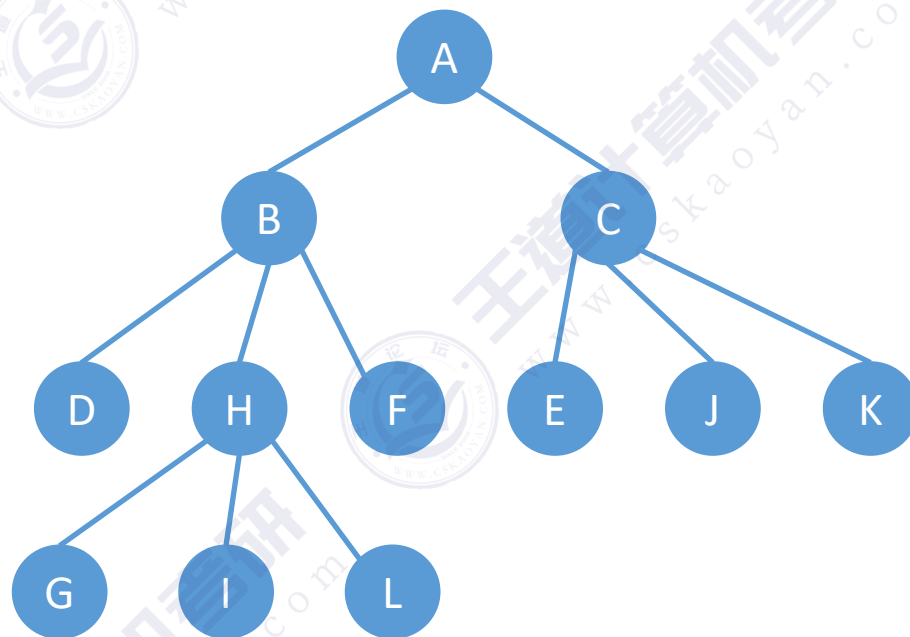
孩子兄弟表示法



二叉树→树的转换



二叉树转换为树



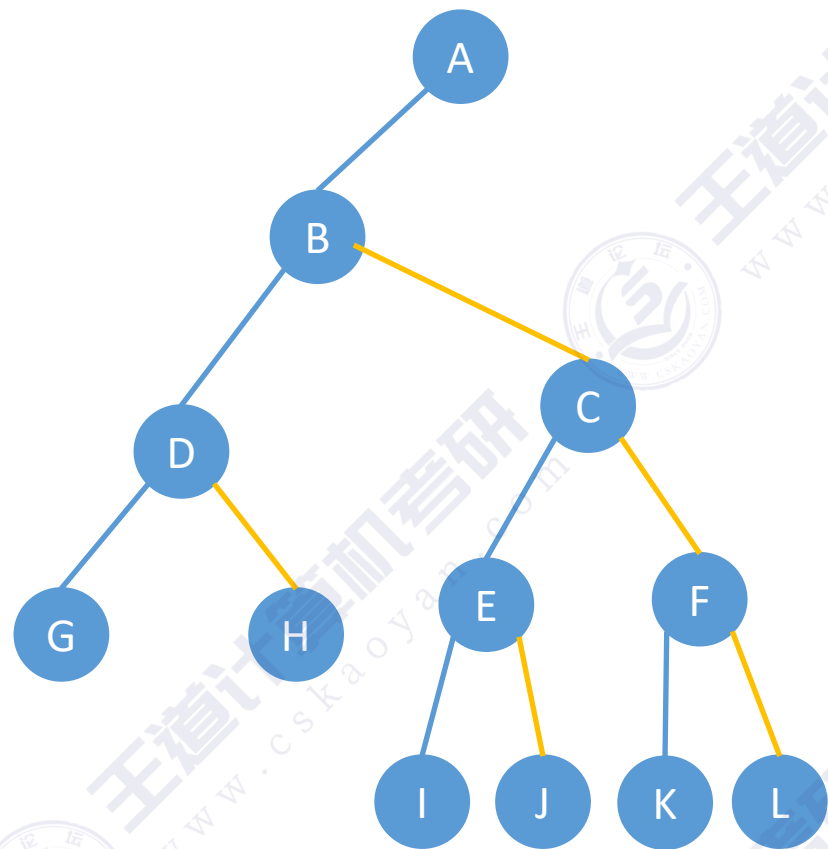
二叉树→树 转换技巧:

①先画出树的根节点

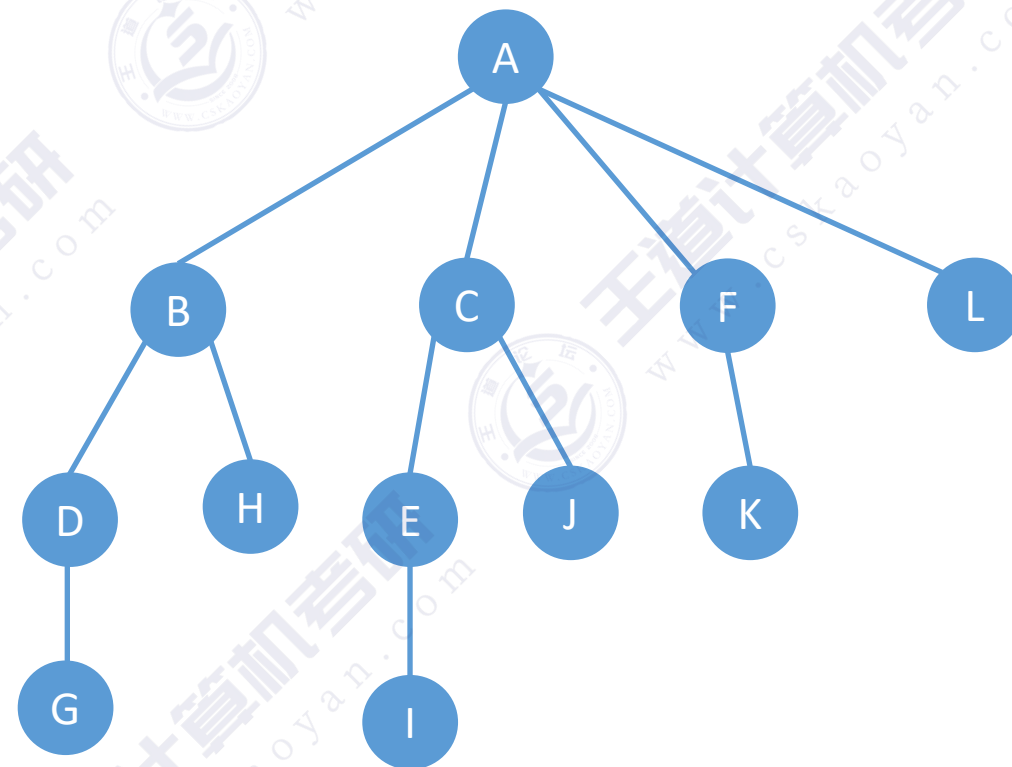
②从树的根节点开始, 按“树的层序”恢复每个结点的孩子

如何恢复一个结点的孩子: 在二叉树中, 如果当前处理的结点有左孩子, 就把左孩子和“一整串右指针糖葫芦”拆下来, 按顺序挂在当前结点的下方

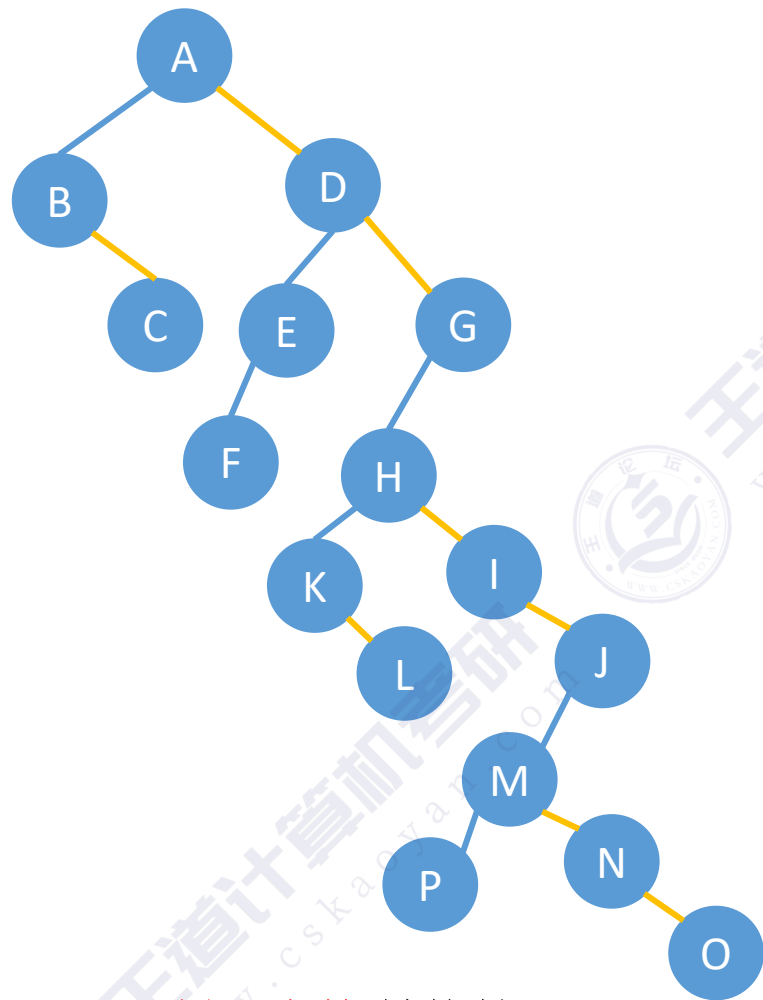
二叉树→树的转换



二叉树转换为树

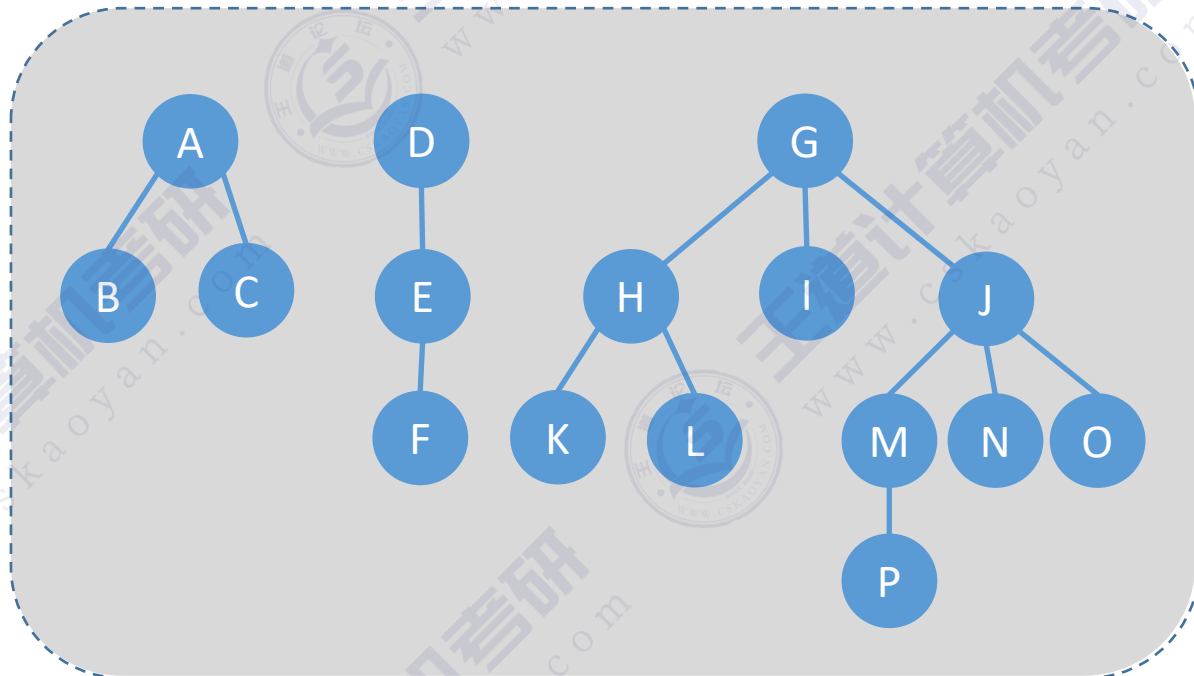


二叉树→森林 的转换



二叉树转森林

森林



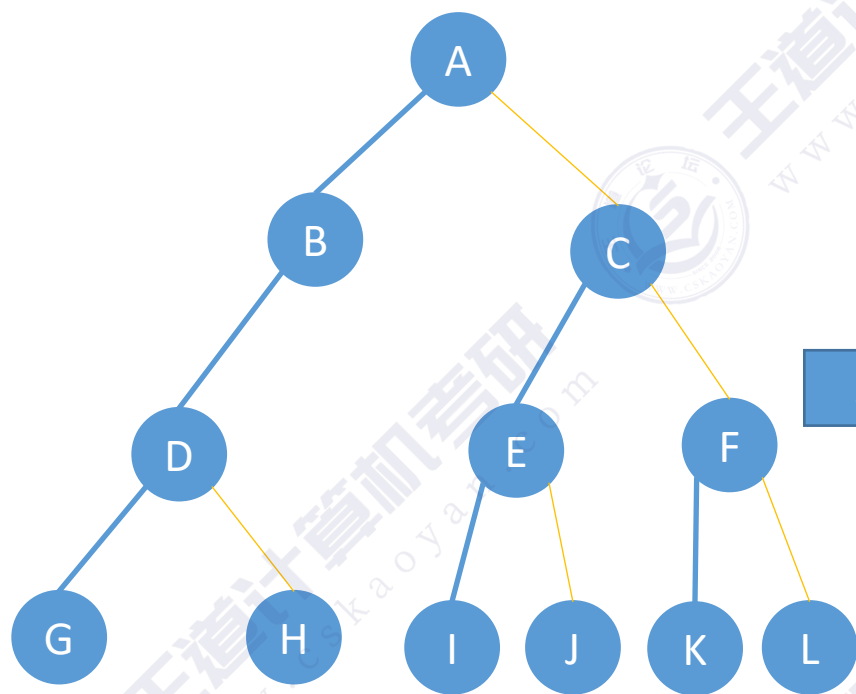
注意：森林中各棵树的根节点视为平级的兄弟关系

二叉树→森林 转换技巧：

- ①先把二叉树的根节点和“一整串右指针糖葫芦”拆下来，作为多棵树的根节点
- ②按“森林的层序”恢复每个结点的孩子

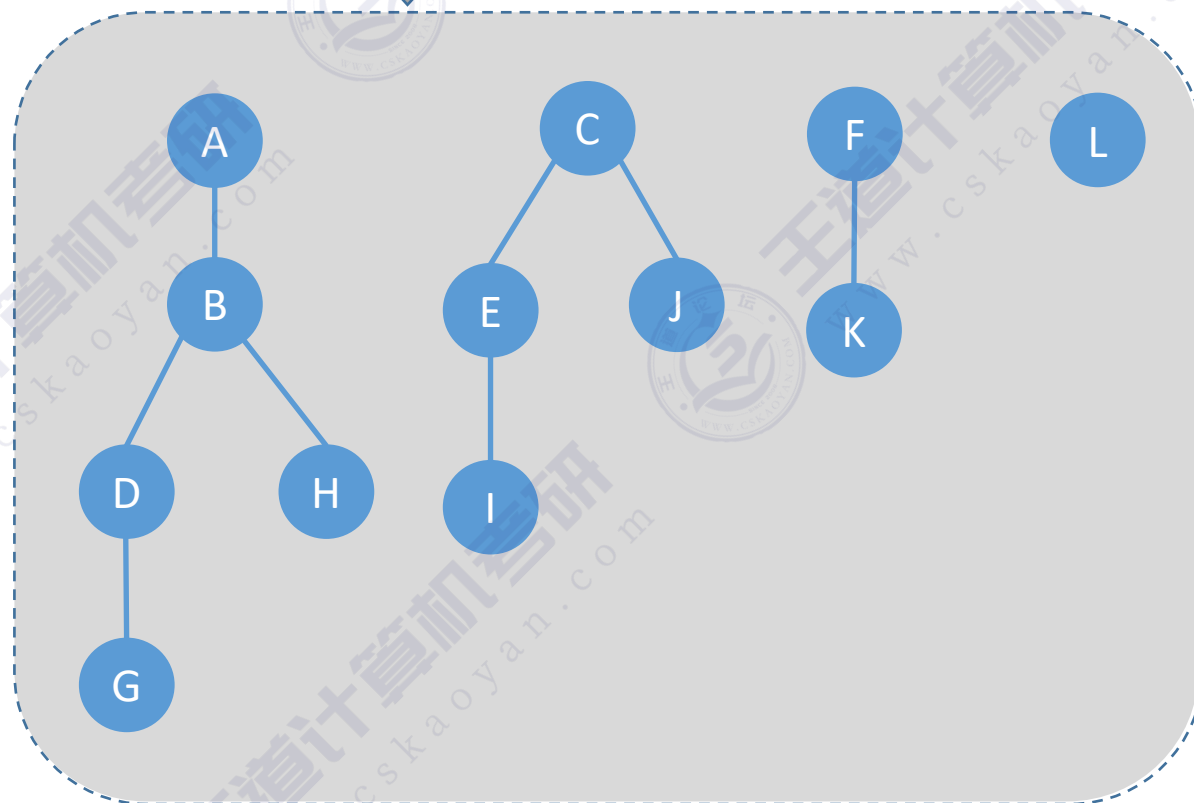
如何恢复一个结点的孩子：在二叉树中，如果当前处理的结点有左孩子，就把左孩子和“一整串右指针糖葫芦”拆下来，按顺序挂在当前结点的下方

二叉树→森林 的转换



二叉树转森林

森林



知识回顾与重要考点

树、森林与二叉树的转换

Key

本质：用孩子兄弟表示法存储树或森林，在形态上与二叉树类似

用孩子兄弟表示法存储森林时，将森林中每棵树的根节点视为平级的兄弟关系

树、森林 转 二叉树

按“层序”依次处理每个结点

处理一个结点的方法是：如果当前处理的结点有孩子，就把所有孩子结点“用右指针串成糖葫芦”，并在二叉树中把第一个孩子挂在当前结点的左指针下方

二叉树 转 树、森林

按“层序”恢复每个结点的孩子

如何恢复一个结点的孩子：在二叉树中，如果当前处理的结点有左孩子，就把左孩子和“一整串右指针糖葫芦”拆下来，按顺序挂在当前结点的下方

Tips: 记“本质”，不记方法。方法只是表象，“本质”才是内核。

本节内容

树、森林 的遍历

知识总览

树、森林的遍历

树的遍历

先根遍历

后根遍历

层序遍历

森林的遍历

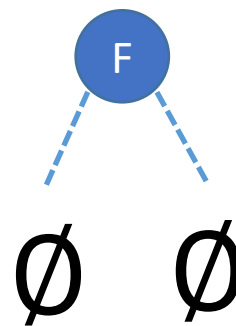
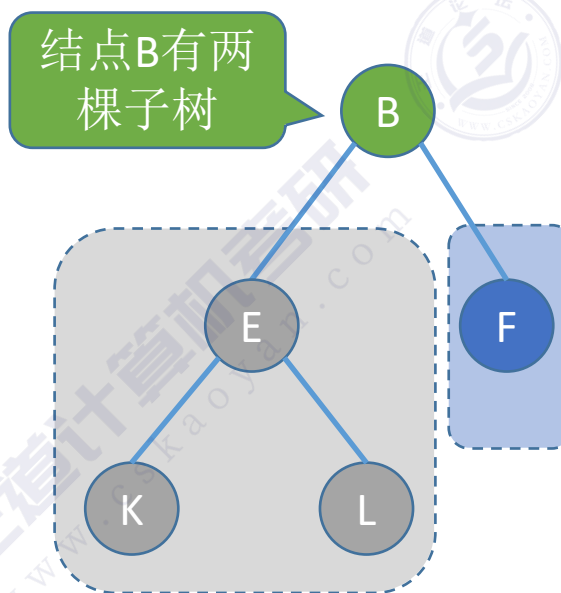
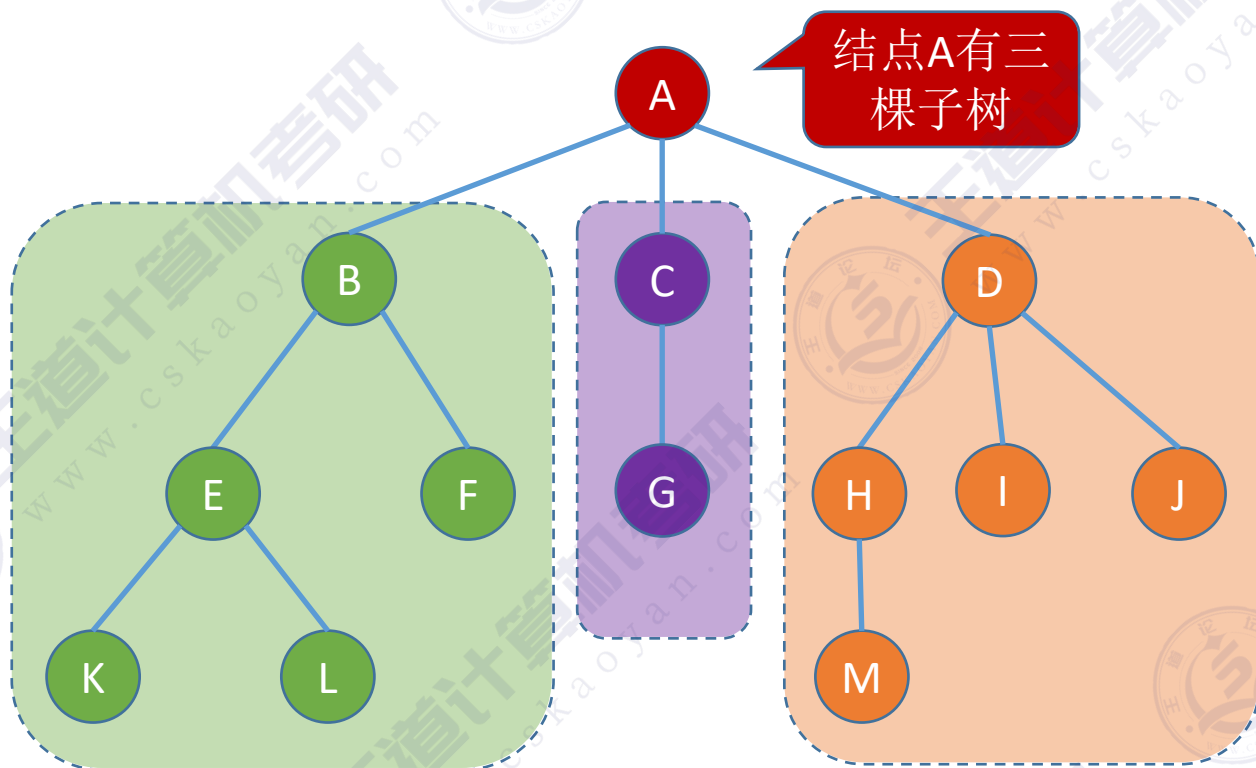
先序遍历

中序遍历

树的逻辑结构

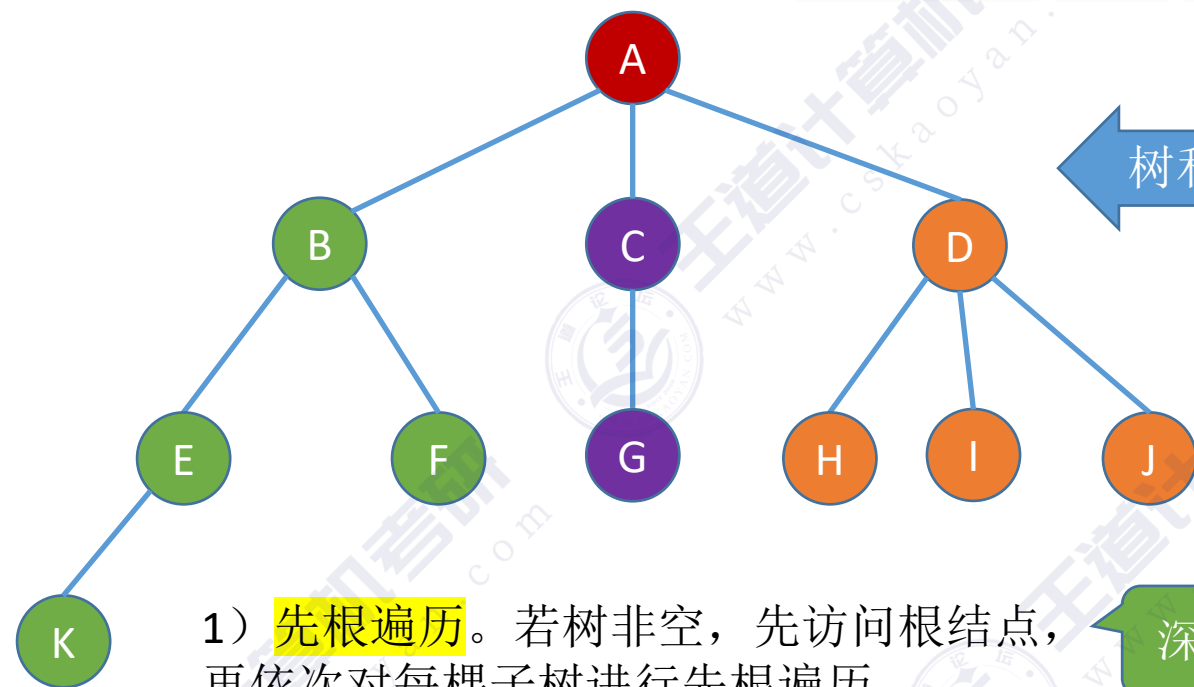
树是 n ($n \geq 0$) 个结点的有限集合, $n = 0$ 时, 称为空树, 这是一种特殊情况。在任意一棵非空树中应满足:

- 1) 有且仅有一个特定的称为根的结点。
- 2) 当 $n > 1$ 时, 其余结点可分为 m ($m > 0$) 个互不相交的有限集合 $T_1, T_2, \dots, T_m$, 其中每个集合本身又是一棵树, 并且称为根结点的子树。

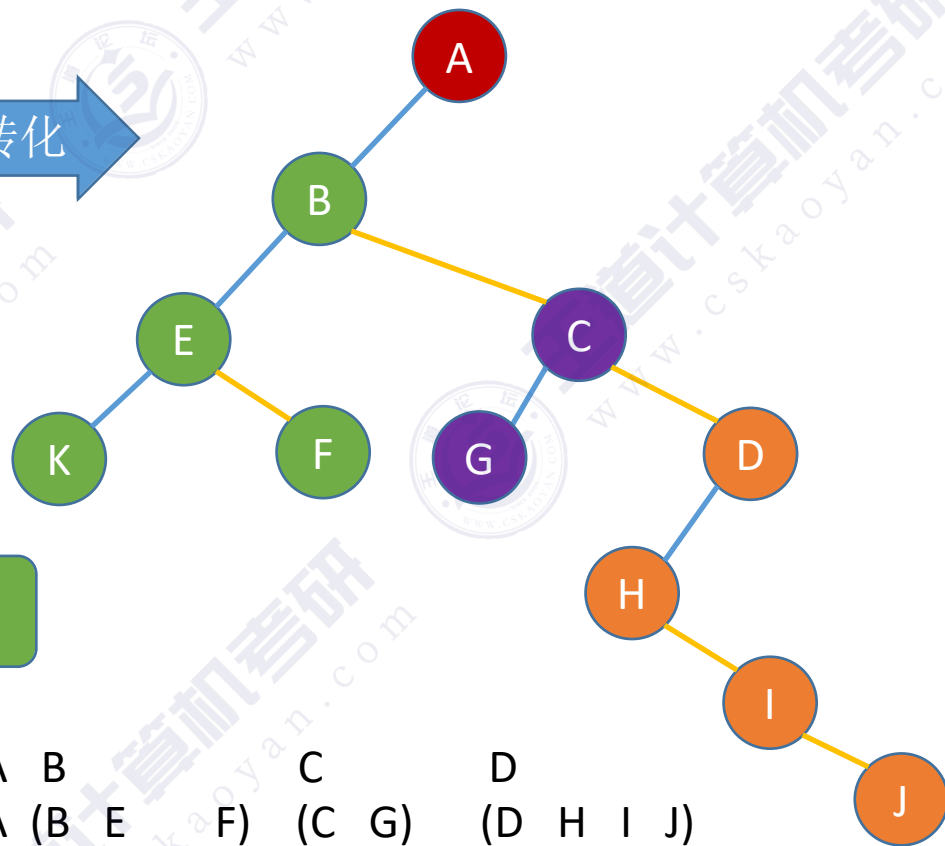


🌲 树是一种递归定义的数据结构

树的先根遍历



树和二叉树的转化



深度优先遍历

1) **先根遍历**。若树非空，先访问根结点，再依次对每棵子树进行先根遍历。

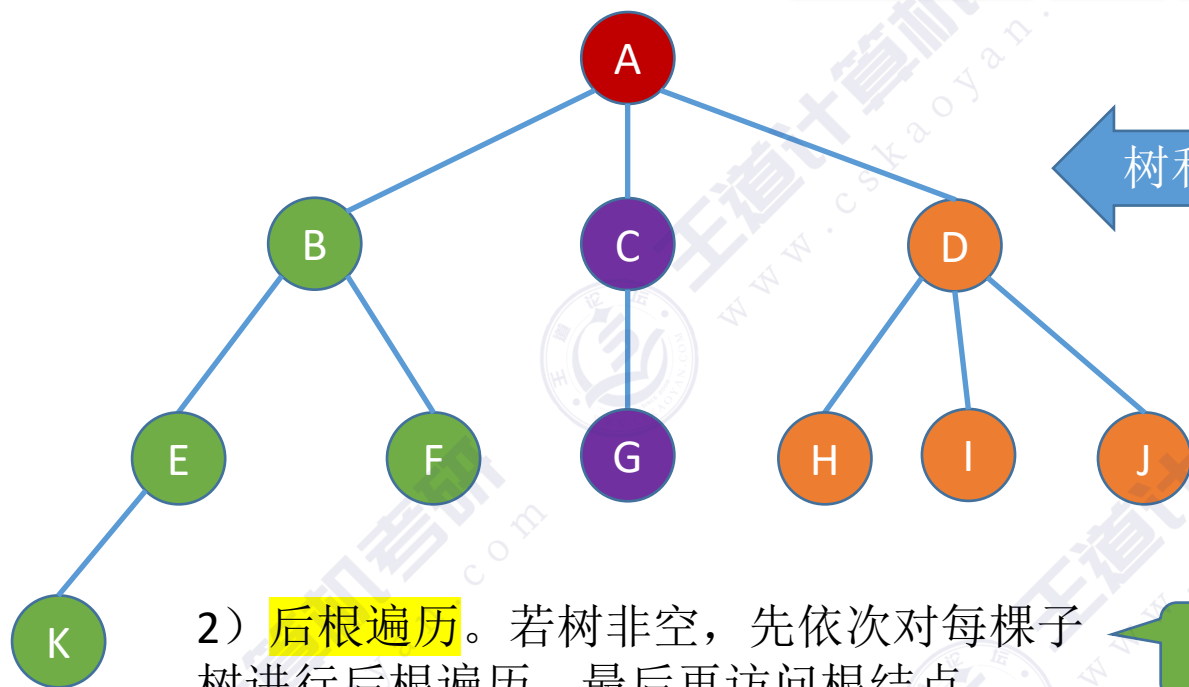
//树的先根遍历

```
void PreOrder(TreeNode *R){  
    if (R!=NULL){  
        visit(R); //访问根节点  
        while(R还有下一个子树T)  
            PreOrder(T); //先根遍历下一棵子树  
    }  
}
```

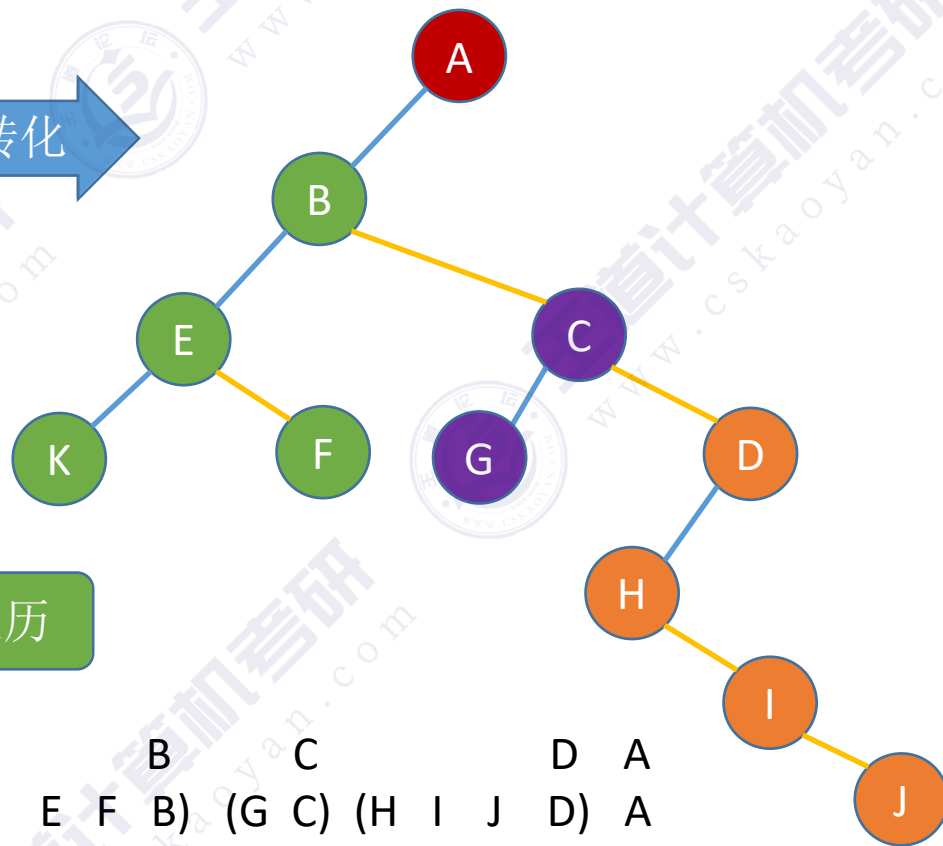
A	B	C	D
A (B E F)	(C G)	(D H I J)	
A (B (E K) F)	(C G)	(D H I J)	

树的先根遍历序列与这棵树相应二叉树的先序序列相同。

树的后根遍历



树和二叉树的转化



深度优先遍历

2) **后根遍历**。若树非空，先依次对每棵子树进行后根遍历，最后再访问根结点。

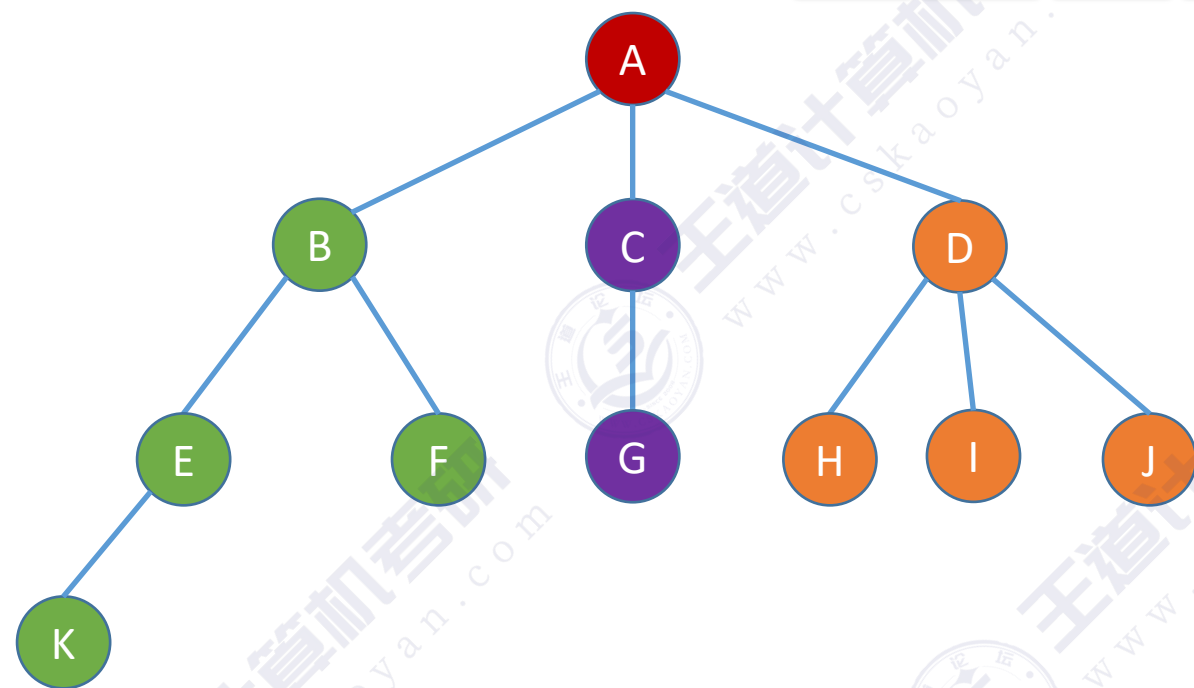
//树的后根遍历

```
void PostOrder(TreeNode *R){  
    if (R!=NULL){  
        while(R还有下一个子树T)  
            PostOrder(T); //后根遍历下一棵子树  
        visit(R); //访问根节点  
    }  
}
```

B C D A
(E F B) (G C) (H I J D) A
((K E) F B) (G C) (H I J D) A

树的后根遍历序列与这棵树相应二叉树的中序序列相同。

树的层次遍历



广度优先遍历

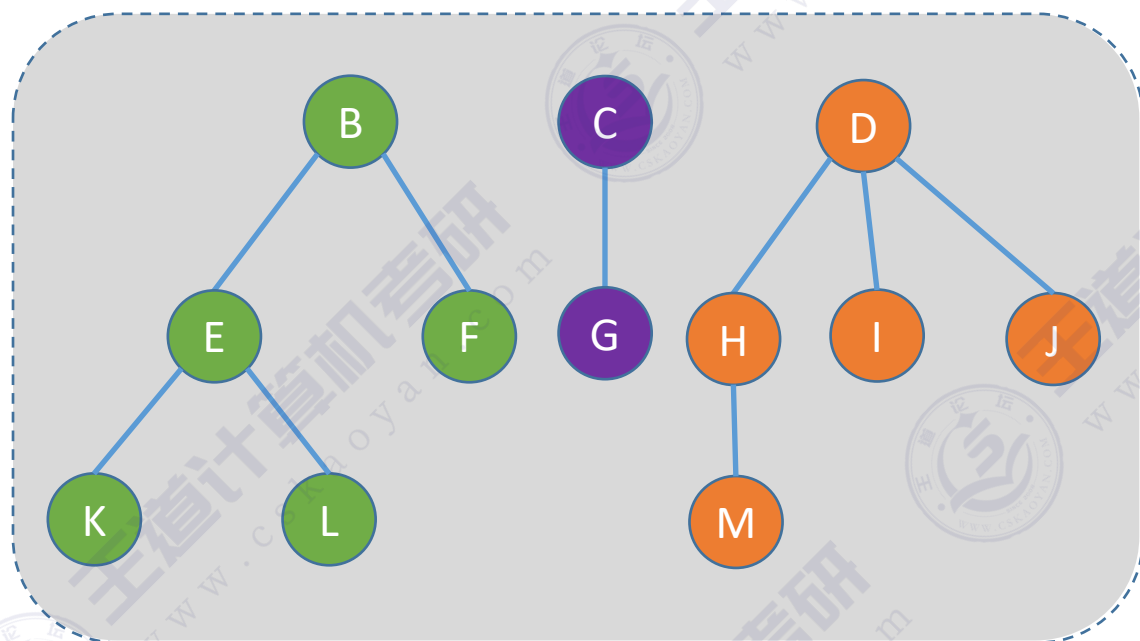
3) 层次遍历 (用队列实现)

- ①若树非空，则根节点入队
- ②若队列非空，队头元素出队并访问，同时将该元素的孩子依次入队
- ③重复②直到队列为空



森林的先序遍历

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合。每棵树去掉根节点后，其各个子树又组成森林。



森林

1) 先序遍历森林。

若森林为非空，则按如下规则进行遍历：

访问森林中第一棵树的根结点。

先序遍历第一棵树中根结点的子树森林。

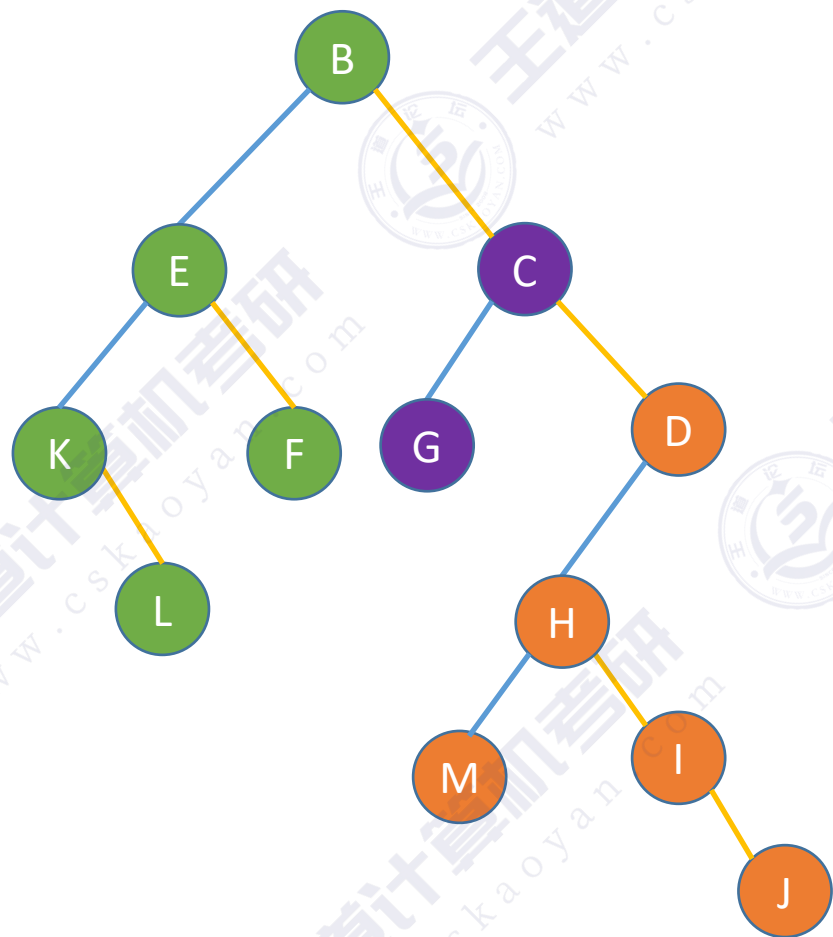
先序遍历除去第一棵树之后剩余的树构成的森林。

B					C					D				
(B	E			F)	(C	G)	(D	H		I	J)			
(B	(E	K	L)	F)	(C	G)	(D	(H	M)	I	J)			

效果等同于依次对各个树进行先根遍历

森林的先序遍历

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合。每棵树去掉根节点后，其各个子树又组成森林。



1) 先序遍历森林。

若森林为非空，则按如下规则进行遍历：

访问森林中第一棵树的根结点。

先序遍历第一棵树中根结点的子树森林。

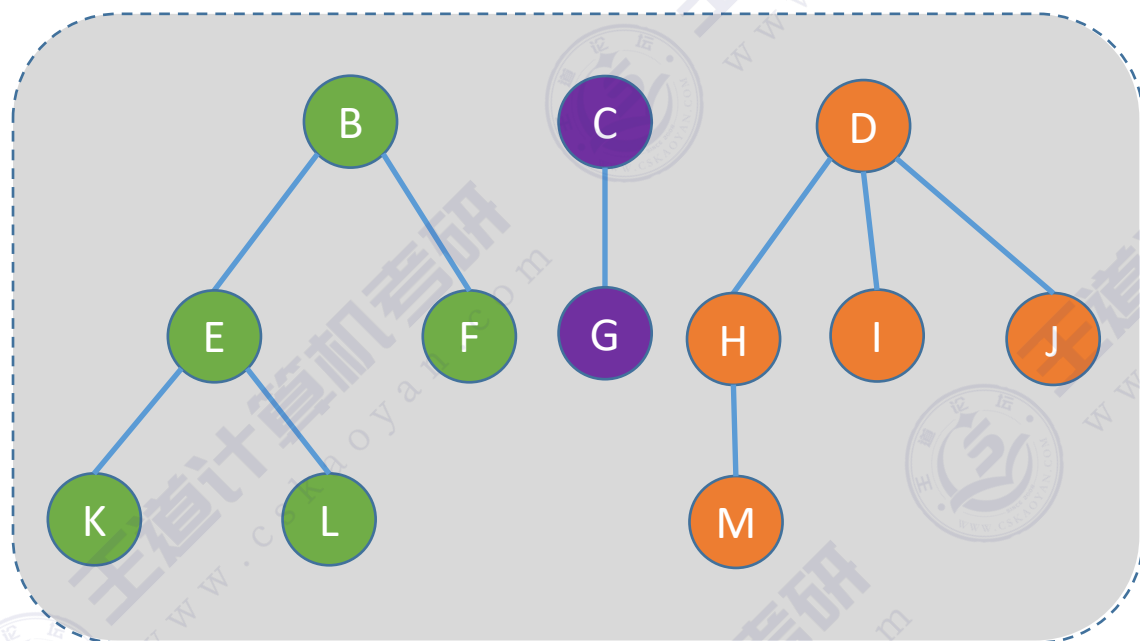
先序遍历除去第一棵树之后剩余的树构成的森林。

B						C		D				
(B	E			F)		(C	G)	(D	H		I	J)
(B	(E	K	L)	F)		(C	G)	(D	(H	M)	I	J)

效果等同于依次对二叉树的先序遍历

森林的中序遍历

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合。每棵树去掉根节点后，其各个子树又组成森林。



森林

2) 中序遍历森林。

若森林为非空，则按如下规则进行遍历：

中序遍历森林中第一棵树的根结点的子树森林。

访问第一棵树的根结点。

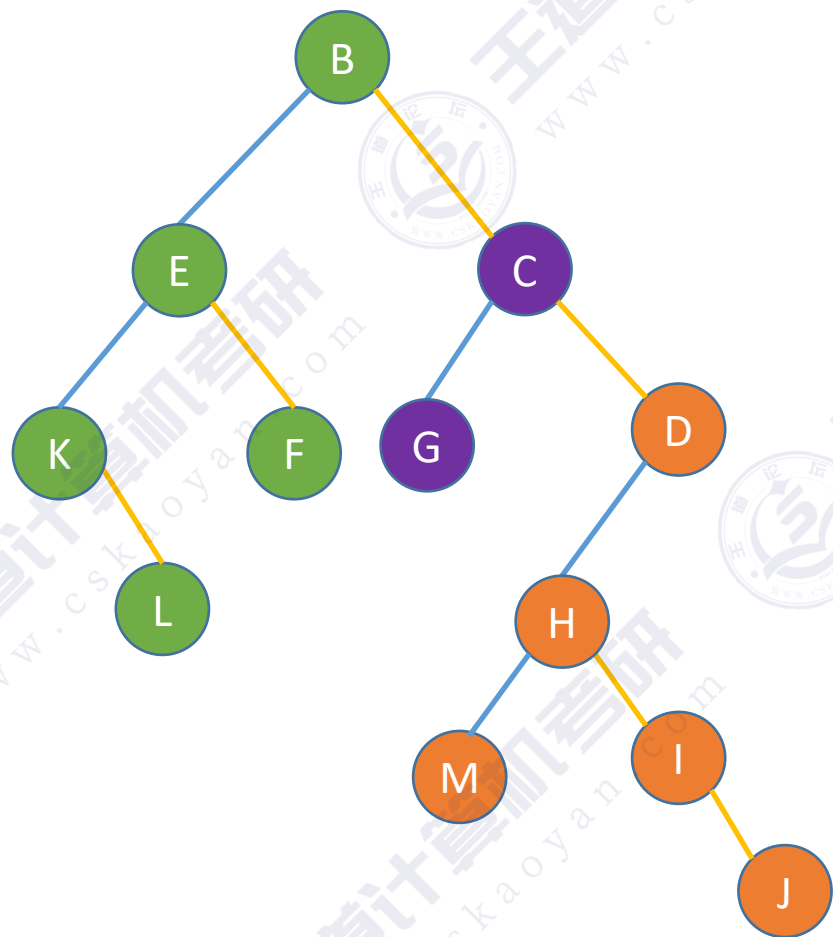
中序遍历除去第一棵树之后剩余的树构成的森林。

```
      B      C      D
(      E  F  B) (G  C) (      H  I  J  D)
((K  L  E)  F  B) (G  C) ((M  H)  I  J  D)
```

效果等同于依次对各个树进行后根遍历

森林的中序遍历

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合。每棵树去掉根节点后，其各个子树又组成森林。



2) 中序遍历森林。

若森林为非空，则按如下规则进行遍历：

中序遍历森林中第一棵树的根结点的子树森林。

访问第一棵树的根结点。

中序遍历除去第一棵树之后剩余的树构成的森林。

B C D
(E F B) (G C) (H I J D)
((K L E) F B) (G C) ((M H) I J D)

效果等同于依次对二叉树的中序遍历

知识回顾与重要考点

树	森林	二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历